Chapter 9

SVM Implementations in Excel

This chapter introduces a user with the basics of solving a machine learning problem using SVM. We have solved problems like XOR, circle and spiral and also shown some of the real world problems like Eigen ear, Contact lens in excel. Many of problems being discussed here are already done through Neural Networks. Hence the reader can refer them in understanding dataset creation and also the preprocessing done for extracting the features.

Note: All implementations of the below explained examples are available in CD. Readers are advised to refer them for better understanding.

9.1 XOR Problem

9.1.1 Problem Def: We have given 4 data points as per the XOR binary operation as shown in figure 9.1.1. The machine has to learn the XOR operation from these inputs and classify all other neighboring points.

Case	X1	X2	Y
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	0

Fig. 9.1.1 XOR Truth Table

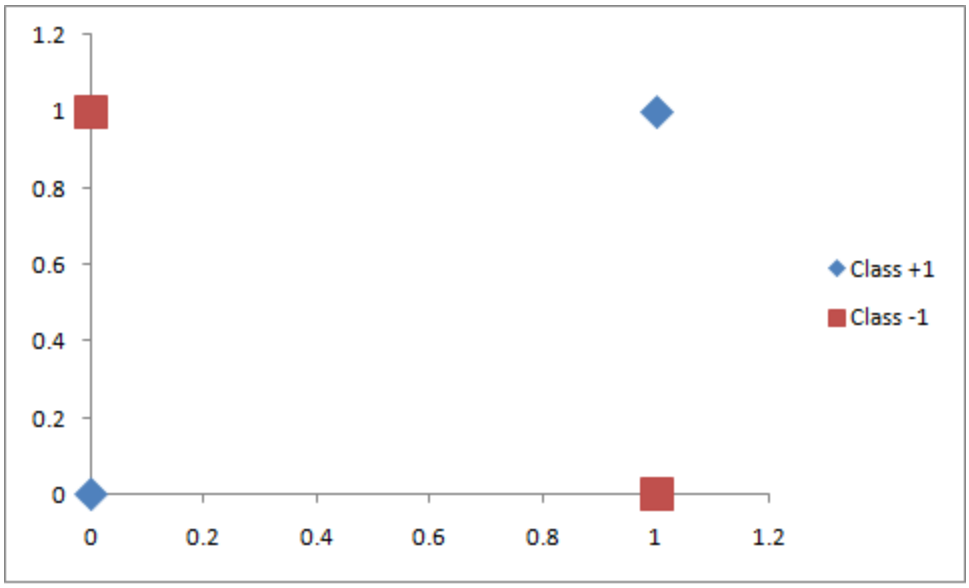


Fig. 9.1.2 XY Plot of input data

As you see in the plot (figure 9.1.2) that there can't be any linear classifier which can separate both the classes. Hence we go for Kernel Methods. Trial and error experience on this data set shows that a polynomial kernel of order 5 will be enough for making the data linearly separable in feature space [1]. Hence $\phi(x)$ is defined as:-

$$\phi(\boldsymbol{x}) = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_1x_2 \\ x_1^2 \\ x_2^2 \end{bmatrix}$$

Mapping each input data point using the mapping function $\phi(x)$ is equivalent to the kernel function $\boldsymbol{K}(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i^T \boldsymbol{x}_j + 1)^2 = \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j)$ .We assumes no explicit bias γ and C= ∞ (infinity). The dual optimization problem is given as shown in Eq. 9.1.1:-

$$\text{Minimize } \mathrm{L_D}(\mathbf{u}) = \frac{1}{2}\mathbf{u}^{\mathrm{T}}\boldsymbol{Q}\mathbf{u} - \mathbf{e}^{\mathrm{T}}\mathbf{u} \qquad (9.1.1)$$

$$\mathbf{0} \le \boldsymbol{u} \le C\boldsymbol{e}$$

Here $\boldsymbol{Q} = \boldsymbol{DKD} = \boldsymbol{K} .*(\boldsymbol{d} * \boldsymbol{d}^{\mathrm{T}})$ where $\boldsymbol{K}$ is the kernel matrix.

Since C= ∞, the constraint reduces to $\boldsymbol{u} \geq \boldsymbol{0}$

Once we have solved the problem, then the prediction function is given by:-

$$f(x) = sign\left(\sum_{i=1}^{4} d_i u_i \phi(x_i)^{\mathrm{T}} \phi(x)\right) \qquad (9.1.2)$$

9.1.2 Steps for solving this in Excel

Step1: Representation of input data in form of matrix 'A', class data as vector 'd' in the sheet name 'Input'. Please note that for two class problem, we represent one class by +1 and other by -1. This is important since the optimization function is designed using these values.

D E F G H I J K L M N O

XOR Problem

Input Data Points			
INDEX	**X1**	**X2**	**CLASS**
1	0	0	1
2	1	1	1
3	1	0	-1
4	0	1	-1

Input Data in Matrix Form			
INDEX	A Matrix		CLASS Vector 'd'
1	0	0	1
2	1	1	1
3	1	0	-1
4	0	1	-1

Fig. 9.1.3 Input Data in excel sheet

Step2: In 'Kernel mapping' sheet we are doing the Kernel mapping operation. At first kernel matrix 'K' is calculated and then using it matrix 'Q' is calculated which is used in Objective function of the optimization problem.

Calculation of matrix 'K':

As you know that Kernel matrix can be calculated either by $\mathrm{F} * \mathrm{F}^{\mathrm{T}}$ where F is the feature matrix. The second way of calculating kernel matrix is through Kernel trick where $\boldsymbol{K}(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i^{T} \boldsymbol{x}_j + 1)^2$ i.e. doing the operation in input space itself. Here the second one is more

computationally efficient as it deals with fewer dimensions. For this example, we use both techniques to calculate the Kernel matrix and show how the later method is more efficient.

As explained before, we are going to use $\phi(x) = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_1x_2 \\ x_1^2 \\ x_2^2 \end{bmatrix}$ for this problem.

For Calculating $F * F^T$, at first we need to calculate feature matrix F. For each input vector, there will be one feature vector. The first element of feature vector is '1' as per our $\phi(x)$. All the others will have the excel formula for calculating them. E.g. the excel formula for second element is =SQRT(2)*Input!$L7. Here Input!$L7 refers to the L7 cell of Input sheet. You will be required only to put formulas in the first row and for remaining you can just drag the formulas from first row to down.

=SQRT(2)*Input!$L7

F

index	1	$\sqrt{2}x_1$	$\sqrt{2}x_2$	$\sqrt{2}x_1 * x_2$	x_1^2	x_2^2
1	1	0	0	0	0	0
2	1	1.41421	1.41421	1.41421	1	1
3	1	1.41421	0	0	1	0
4	1	0	1.41421	0	0	1

Fig. 9.1.4 Feature matrix

Here index represents each data point. For calculating $K = F * F^T$. In excel we will use the formula {=MMULT(B20:G23,TRANSPOSE(B20:G23))} where range B20:G23 refers to matrix 'F' (Refer matrix portion of chapter 02 in case of difficulty with matrix operations in excel) .

Let's now see how we can calculate K using the function $K(x_i, x_j) = (x_i^T x_j + 1)^2$. Here we will use excel's What-If analysis (chapter 02) which will help for doing iteration over 'i' and 'j' variables in $K(x_i, x_j)$.

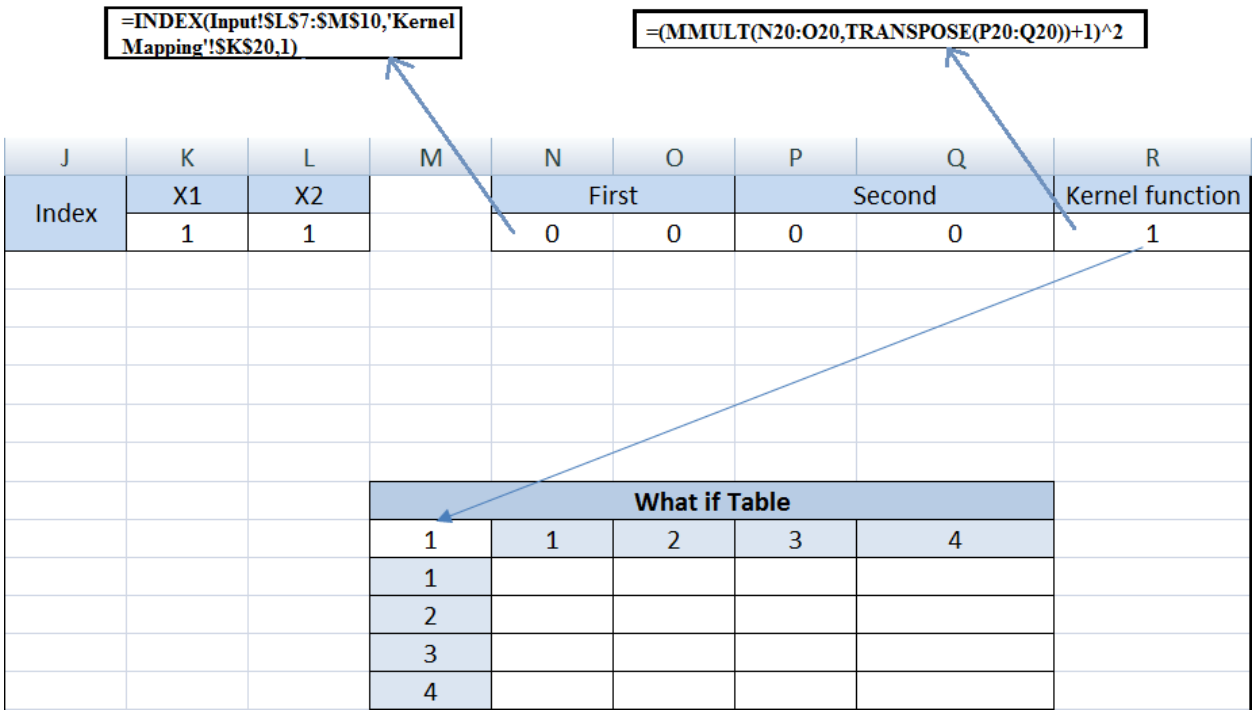


Fig. 9.1.5 What- if data table operation (a)

For this we create two cells designated as X1 and X2 as shown in figure 9.1.5. When X1 have value as 1 then the two cells designated as First will contain the input vector at index 1. Similarly the cells called Second will have the input vector at index given by X2. The excel function used for this is Index().

The Kernel function value for a particular i, j is calculated by formula $(\boldsymbol{x}_i^T \boldsymbol{x}_j + 1)^2$. The excel formula will be =(MMULT(N20:O20,TRANSPOSE(P20:Q20))+1)^2.

In what-if table, the first row and first column contain the index number of each input data and Kernel function is copied to the first cell of the table. Here it is cell address 'M28'. Now we will run what-if analysis as shown in figure 9.1.6 so that it calculates the kernel matrix 'K'. The row input and column input cell for what-if will be X1 and X2 respectively.

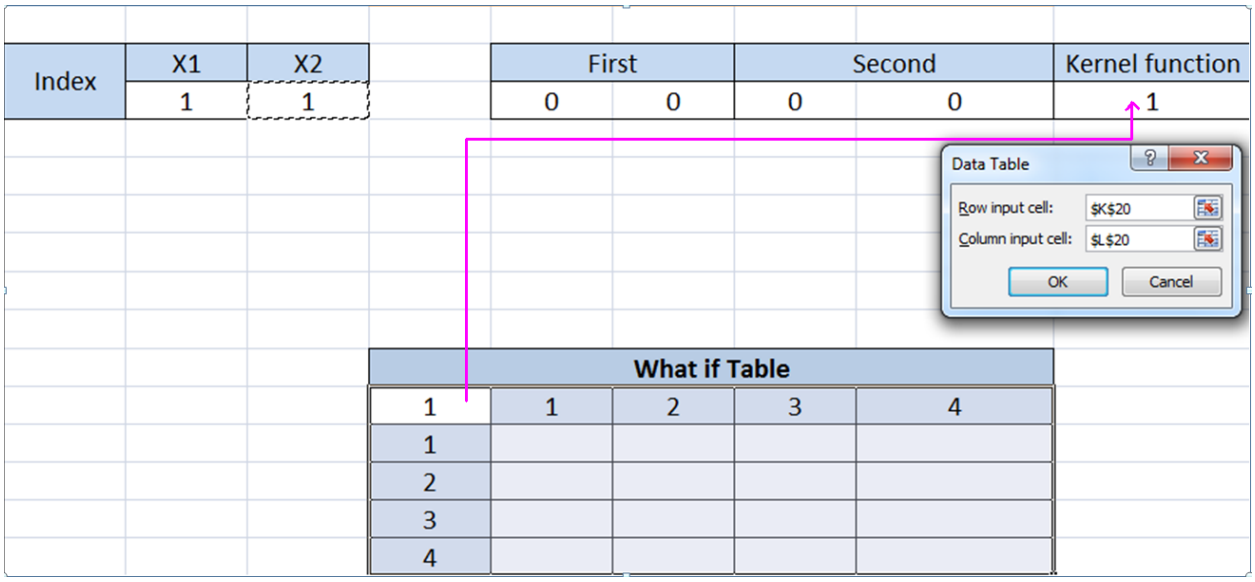


Fig. 9.1.6 What- if data table operation (b)

After running what-if analysis

What if Table				
1	1	2	3	4
1	1	1	1	1
2	1	9	4	4
3	1	4	4	1
4	1	4	1	4

Fig. 9.1.7 Kernel Matrix Calculation

Here we see that the kernel matrices being calculated from $\mathrm{F}*\mathrm{F}^{\mathrm{T}}$ and through kernel trick are same but later one is more computational efficient while handling big matrices and higher dimensions.

Step3: Calculation of Q matrix.

In excel you can see Q matrix being calculated using the kernel matrix (K) that we got in step2. We have used the formula $Q = DKD = K.*(d*d^T)$. Here 'D' is diagonal matrix having diagonal values as class values in the order as given as the input data. We have shown both D matrix and also $d*d^T$ calculation in the excel sheet. You can take one of the ways to calculate the Q matrix. If you go for $Q = DKD$ then it is normal matrix multiplication using MMULT function of excel. But if you go for $Q = K.*(d*d^T)$ then .* represents element by element multiplication for which we should not use MMULT. It will be just a normal multiplication operation.

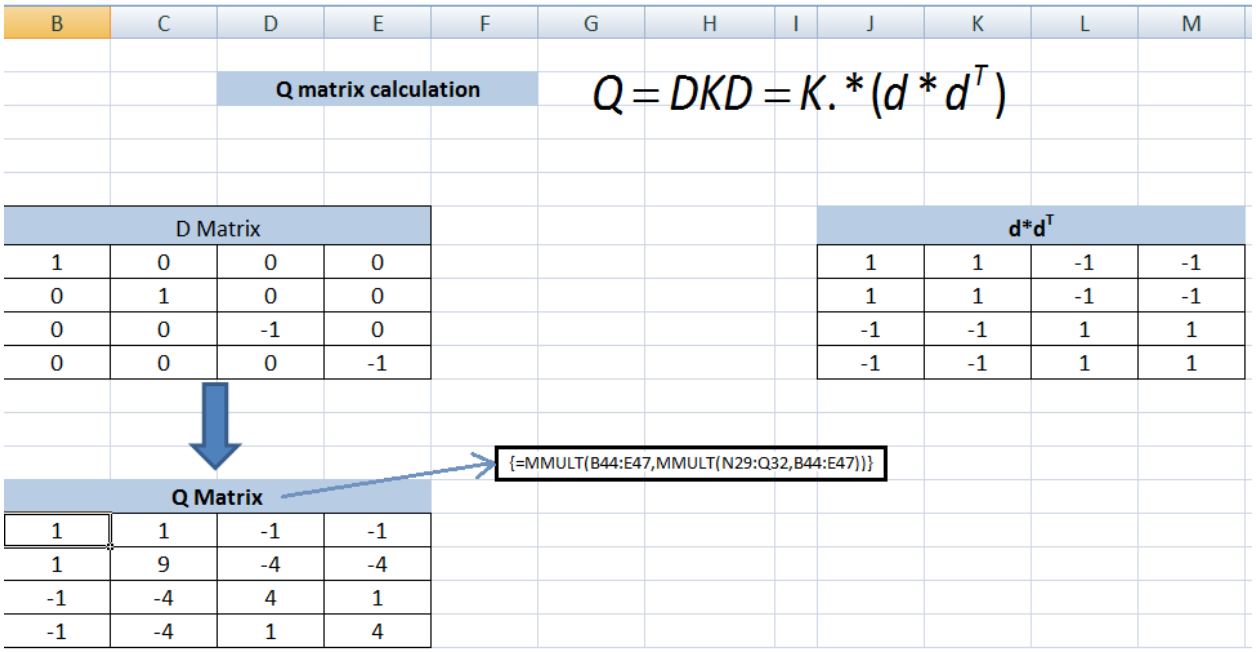


Fig. 9.1.8 Q matrix calculation

Step4: Optimization

Now we are in optimization stage and objective function is given as below:-

$$\text{Minimum } L_D(\mathbf{u}) = \frac{1}{2}\mathbf{u}^T\boldsymbol{Q}\mathbf{u} - \mathbf{e}^T\mathbf{u}$$

$$\textbf{subject to } \mathbf{u} \geq \mathbf{0}$$

In Optimization sheet you can see that the excel formula used is

=(0.5*(MMULT(MMULT(TRANSPOSE(Optimization!$J$20:$J$23),'KernelMapping'!$B$52:$E$55),Optimization!$J$20:$J$23)))-1*MMULT(TRANSPOSE($K$20:$K$23),J20:J23)

The U and e vectors are initialized to 1.

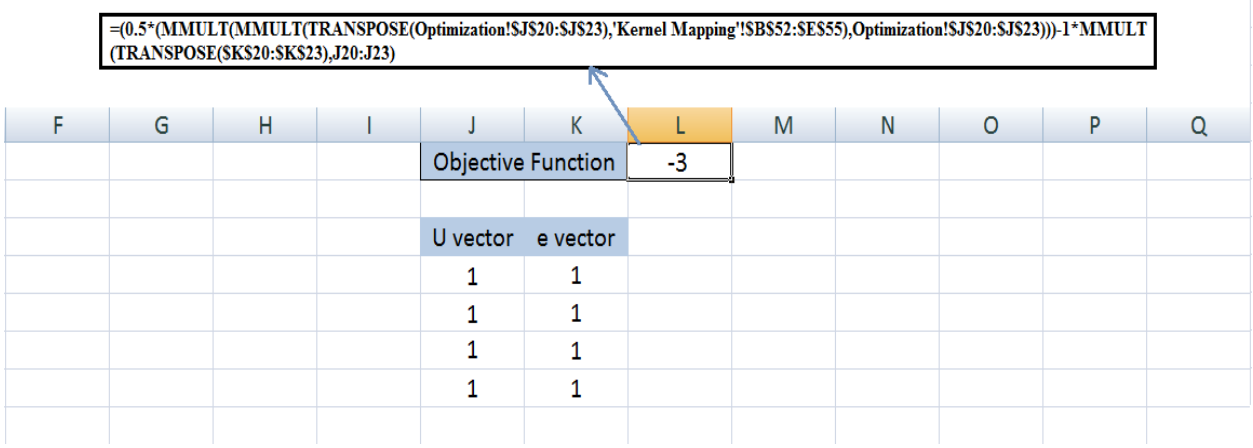


Fig. 9.1.9 Objective function and constraints for optimization

Step5: Running solver

As you already know that solver is optimization tool inbuilt in excel. We will be using Solver for solving the optimization problem shown figure 9.1.9.

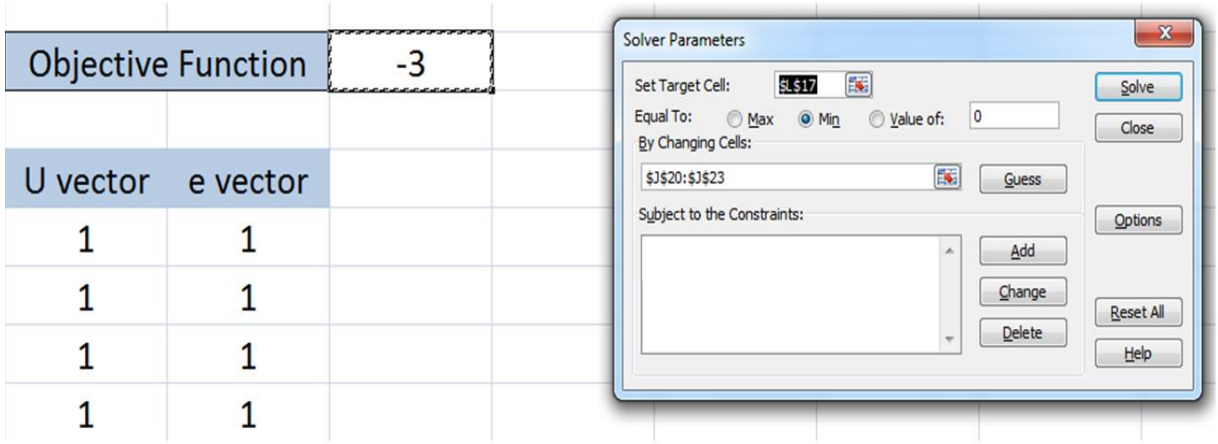


Fig. 9.1.10 Solver parameters

We will set target cell (figure 9.1.10) as 'L17' where we have defined the objective function. 'Equal to' parameter will be set as 'Min'. Changing cells will be given as the range address of U vector and constraints (figure 9.1.11) will also be given as the elements of U vector greater that zero.

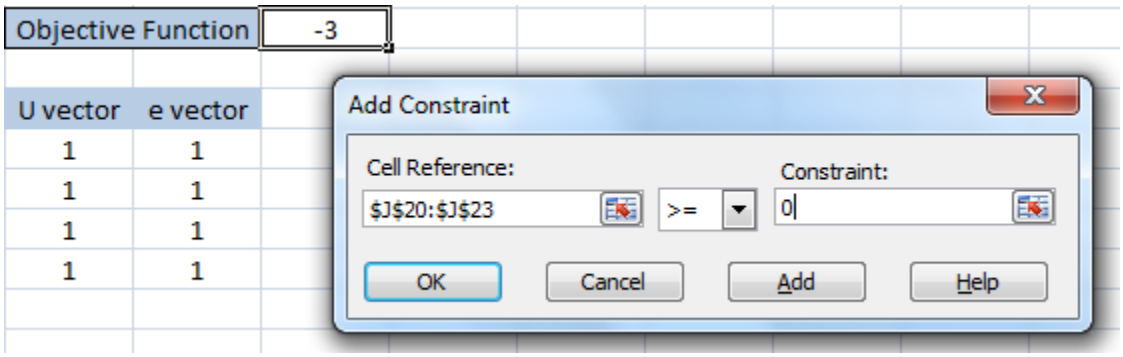


Fig. 9.1.11 Adding Constraints

After giving all these parameters we will run the solver. The solver will give optimized values as shown in figure 9.1.12.

Objective Function	-5.83333
U vector	e vector
4.333321	1
1.999996	1
2.666661	1
2.666661	1

Fig. 9.1.12 Solver Output

Step6: Prediction Function

The optimal solution obtained is

$u_1 = 4.3333,\ u_2 = 2.0000, u_3 = 2.6667, u_4 = 2.6667$

As explained earlier the prediction function is given by the following function. $f(x) = sign\left(\sum_{i=1}^{4} d_i u_i \phi(x_i)^{\mathrm{T}} \phi(x)\right)$. Here 'i' will iterate for 'm' training data points. $\phi(x)$ stands for the new test data point for which the predicted class value will be $f(x)$.

$$\sum_{i=1}^{4} d_i u_i \phi(\boldsymbol{x}_i) = \left[4.333 \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + 2 \begin{bmatrix} 1 \\ \sqrt{2} \\ \sqrt{2} \\ \sqrt{2} \\ 1 \\ 1 \end{bmatrix} - 2.6667 \begin{bmatrix} 1 \\ \sqrt{2} \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} - 2.6667 \begin{bmatrix} 1 \\ 0 \\ \sqrt{2} \\ 0 \\ 0 \\ 1 \end{bmatrix} \right] = \begin{bmatrix} 1 \\ -0.9429 \\ -0.9429 \\ 2.8284 \\ -0.6667 \\ -0.6667 \end{bmatrix} \qquad (9.1.3)$$

In excel the operations shown in eq. 9.1.3 is implemented as shown in figure 9.1.13:-

index	Feature Space Vectors						Class	U vector
1	1	0	0	0	0	0	1	4.333321
2	1	1.4142136	1.414214	1.414214	1	1	1	1.999996
3	1	1.4142136	0	0	1	0	-1	2.666661
4	1	0	1.414214	0	0	1	-1	2.666661

=O12*M12*F12:K12

i						
1	4.333321	0	0	0	0	0
2	1.999996	2.828421	2.828421	2.828421	1.999996	1.999996
3	-2.66666	-3.77123	0	0	-2.66666	0
4	-2.66666	0	-3.77123	0	0	-2.66666
Summation	0.999995	-0.94281	-0.94281	2.828421	-0.66666	-0.66666

Fig. 9.1.13 Prediction function

Once $\sum_{i=1}^{4} d_i u_i \phi(\boldsymbol{x}_i)$ is calculated, we go for predicting each data point present in the grid. The grid range is taken from -2 to +2, in both X1 and X2 direction. We will divide -2 to +2 in 41 equal parts. Hence the delta change between adjacent values will be 0.1.

For calculating the prediction value for each data point present in the grid, we will again use What-if analysis. At first, each input test vector is converted into corresponding feature vector using $\phi(x)$ mapping function. The feature vector is then multiplied with the summation vector as obtained in figure 9.1.3. This is represented as $f(x) = sign\left(\sum_{i=1}^{4} d_i u_i \phi(x_i)^{\mathrm{T}} \phi(x) \right)$. The sign value will tell whether that data point belongs to Class -1 or Class +1. The excel snapshots are shown in the next Figures. In figure 9.1.14, the index cell will have X1 and X2 of the test data point. The

Feature vector is calculated from the same old formula. The prediction cell will do the multiplication of summation vector with new feature vector and get its sign. This sign value is copied to the first cell of the What-if table (figure 9.1.15).

i						
1	4.333321	0	0	0	0	0
2	1.999996	2.828421	2.828421	2.828421	1.999996	1.999996
3	-2.66666	-3.77123	0	0	-2.66666	0
4	-2.66666	0	-3.77123	0	0	-2.66666
Summation	0.999995	-0.94281	-0.94281	2.828421	-0.66666	-0.66666

Index	x1	x2
	2	-2

Feature Vector	1	2.828427	-2.82843	-5.65685	4	4

Prediction	-1

=SIGN(MMULT(H25:M25,TRANSPOSE(H30:M30)))

Fig. 9.1.14 What-if table inputs

Running What-If analysis

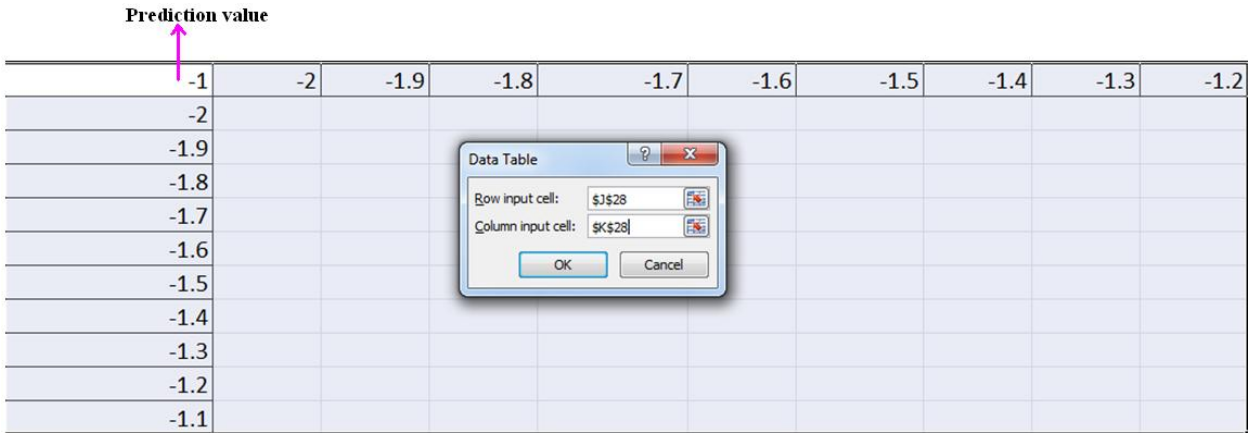


Fig. 9.1.15 What-if table parameters

This will fill all the grid points with their corresponding prediction value.

	A	B	C	D	E	F	G	H
37	-1	-2	-1.9	-1.8	-1.7	-1.6	-1.5	-1.4
38	2	-1	-1	-1	-1	-1	-1	-1
39	1.9	-1	-1	-1	-1	-1	-1	-1
40	1.8	-1	-1	-1	-1	-1	-1	-1
41	1.7	-1	-1	-1	-1	-1	-1	-1
42	1.6	-1	-1	-1	-1	-1	-1	-1
43	1.5	-1	-1	-1	-1	-1	-1	-1
44	1.4	-1	-1	-1	-1	-1	-1	-1
45	1.3	-1	-1	-1	-1	-1	-1	-1
46	1.2	-1	-1	-1	-1	-1	-1	-1
47	1.1	-1	-1	-1	-1	-1	-1	-1

Fig. 9.1.16 What table analysis output

Step7: Plotting

Finally we reach to the plotting stage to see our results. Plotting can be achieved in two ways. One is using Macro code and other one is without macro (figure 9.1.17) (using Conditional formatting). In plotting sheet, at cell A1 we have put the formula {=Prediction!B38} so that we can copy B38 cell from prediction sheet (B38 cell is the first cell of the prediction table obtained from step6). After this you can drag the A1 cell from A1:AO41 so that the entire prediction table is brought inside the plotting sheet. Now select the entire table and click the conditional formatting button present in the Home tab > Styles group, click onto the colour scales and choose the colour you wish. We have chosen the Blue-Yellow-Red combination. Fig 9.1.17 shows the usage of Conditional Formatting.

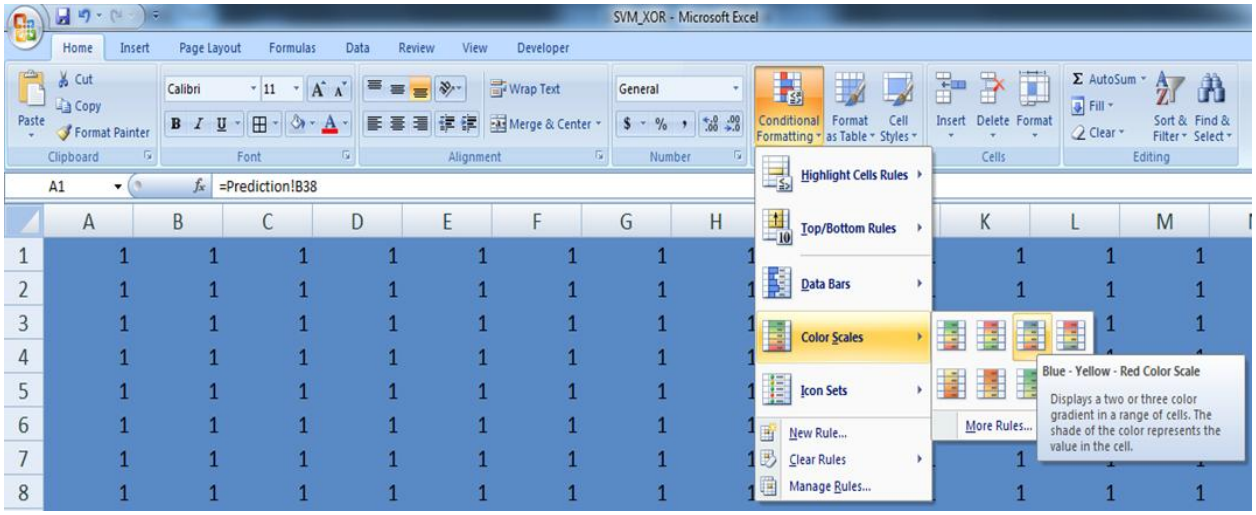


Fig. 9.1.17 Plotting using conditional formatting

To see the entire plotted image in same screen with more clarity, we will reduce the Row height and column width for each cell (figure 9.1.18). This is achieved by selecting the entire table and clicking on the Format cell button present in the cells group of Home tab. Use the row height as 4 and column width as 0.4. Now you can see the image as shown figure 9.1.18 being plotted in your sheet.



Fig. 9.1.18 Final plot image

Understanding the plot image

The image that you can see shows how our machine has learned the XOR problem. As we said earlier, in XOR problem there is no linear classifier which can separate both the classes. Hence we used kernel methods and applied polynomial kernel of order 5.The image shown in figure 9.1.18 tells that SVM has given one class as the central diagonal and remaining for the other class (shown in blue and red color respectively). This exactly matches to what we humans will think for solving XOR problem. Though we had some additional steps for solving through SVM but the concept is simple. The concept is: if our input data is not separable in input space, map it to the feature space and classify it linearly. The linear classifier when mapped back to input space gives a smooth non-linear plane in the input space.

9.2 Circle Problem

9.2.1 Problem Def: This is also a binary classifier problem where the data is distributed in the form of concentric circles. The inner circles belong to Class -1 while outer circles belong to Class +1. The machine has to learn these circular datasets and classify all the neighboring points.

Figure 9.2.1 is the plot showing two classes.

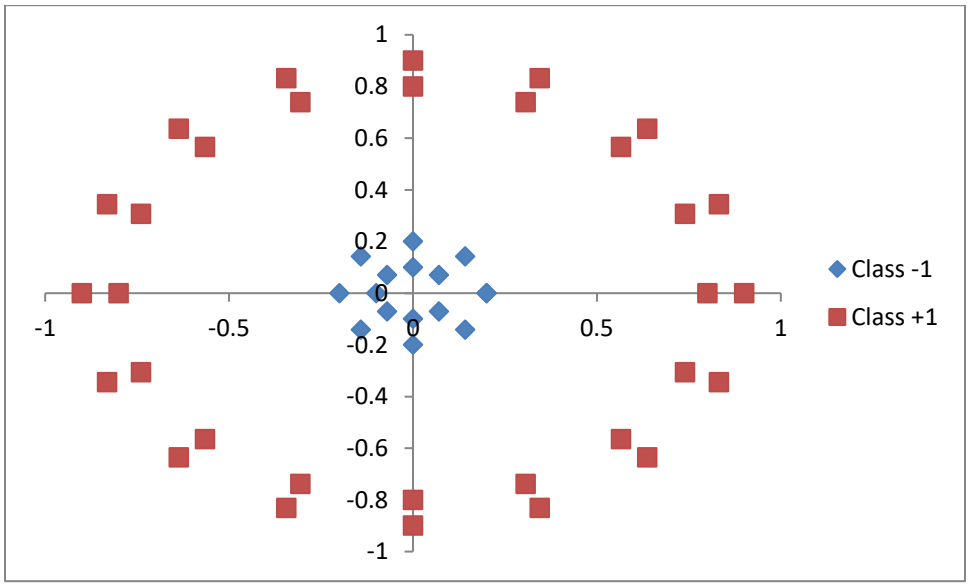


Fig. 9.2.1 XY Plot of input data

Before we start, let us apply our common sense and predict what might be the classifier? Ideally there should be a circular boundary which should separate the two classes. Let us see whether SVM also tells the same answer.

9.2.2 Steps for solving this in Excel

Step1: Understanding the dataset.

This is exactly same as we discussed in Neural Networks (Chapter 5). The only change here is that we have used two concentric circles for each class. The inner circles are of radii 1 and 2; the outer circles are of radii 8 and 9. One important thing that we have done here is that we have normalized each of the data point (X1 and X2).Normalizing is done to avoid problems which arise when the value of kernel become too small or too big. Here we have used the normalizing factor as 10. Hence we have divided each data point with 10 so that its variation is limited. Figure 9.2.2 is the snapshot taken from Input sheet.

Input Data Points					
Index	Radius	Theta	X1	X2	Class
1	1	0.785398	0.070711	0.070711	-1
2	1	1.570796	6.13E-18	0.1	-1
3	1	2.356194	-0.07071	0.070711	-1
4	1	3.141593	-0.1	1.23E-17	-1
5	1	3.926991	-0.07071	-0.07071	-1
6	1	4.712389	-1.8E-17	-0.1	-1
▪	▪	▪	▪	▪	▪
8	2	6.283185	0.2	-4.9E-17	-1
9	2	7.068583	0.141421	0.141421	-1
10	2	7.853982	6.13E-17	0.2	-1
11	2	8.63938	-0.14142	0.141421	-1
▪	▪	▪	▪	▪	▪
19	8	4.31969	-0.30615	-0.7391	1
20	8	4.712389	-1.5E-16	-0.8	1
21	8	5.105088	0.306147	-0.7391	1
22	8	5.497787	0.565685	-0.56569	1
▪	▪	▪	▪	▪	▪
33	9	0.392699	0.831492	0.344415	1
34	9	0.785398	0.636396	0.636396	1
35	9	1.178097	0.344415	0.831492	1
36	9	1.570796	5.51E-17	0.9	1

Fig. 9.2.2 Input Data in excel sheet

Step2: Calculation of K and Q matrices.

We have used the same polynomial kernel as used in XOR problem. The Kernel matrix is calculated using kernel function $(\boldsymbol{x}_i^T \boldsymbol{x}_j + 1)^2$. Figure 9.2.3 shows the snapshot of the K matrix being calculated from What-IF analysis.

	J	K	L	M	N	O	P	Q	R
13									
14	Index	X1	X2		First		Second		Kernel function
15		2	2		6.13E-18	0.1	6.12574E-18	0.1	1.0201
16									
17									
18	What if Table								
19	1.0201	1	2	3	4	5	6	7	8
20	1	1.0201	1.014192	1	0.985908	0.9801	0.985907864	1	1.028484271
21	2	1.014192	1.0201	1.014192	1	0.985908	0.9801	0.985907864	1
22	3	1	1.014192	1.0201	1.014192	1	0.985907864	0.9801	0.971915729
23	4	0.985908	1	1.014192	1.0201	1.014192	1	0.985907864	0.9604
24	5	0.9801	0.985908	1	1.014192	1.0201	1.014192136	1	0.971915729
25	6	0.985908	0.9801	0.985908	1	1.014192	1.0201	1.014192136	1
26	7	1	0.985908	0.9801	0.985908	1	1.014192136	1.0201	1.028484271
27	8	1.028484	1	0.971916	0.9604	0.971916	1	1.028484271	1.0816
28	9	1.0404	1.028484	1	0.971916	0.9604	0.971915729	1	1.057368542
29	10	1.028484	1.0404	1.028484	1	0.971916	0.9604	0.971915729	1
30	11	1	1.028484	1.0404	1.028484	1	0.971915729	0.9604	0.944231458
31	12	0.971916	1	1.028484	1.0404	1.028484	1	0.971915729	0.9216
32	13	0.9604	0.971916	1	1.028484	1.0404	1.028484271	1	0.944231458
33	14	0.971916	0.9604	0.971916	1	1.028484	1.0404	1.028484271	1
34	15	1	0.971916	0.9604	0.971916	1	1.028484271	1.0404	1.057368542

Fig. 9.2.3 Kernel Matrix Calculation

For calculating Q, we have used the formula $Q = DKD = K.*(d*d^T)$.

Step3: Optimization using Solver

The optimization function is given as:-

$$\text{Minimum } L_D(\mathbf{u}) = \frac{1}{2}\mathbf{u}^T\boldsymbol{Q}\mathbf{u} - \mathbf{e}^T\mathbf{u}$$

$$\textbf{subject to } \mathbf{u} \geq \mathbf{0}$$

In Optimization sheet (figure 9.2.4) you can see that the excel formula used as:-

=(0.5*(MMULT(MMULT(TRANSPOSE(Optimization!$J$20:$J$67),'Kernel Mapping'!$B$128:$AW$175),Optimization!$J$20:$J$67)))-1*MMULT(TRANSPOSE($K$20:$K$67),$J$20:$J$67)

The U and e vectors are initialized to 1.

Objective Function		2.568893
U vector	e vector	
1	1	
1	1	
1	1	
1	1	
1	1	
1	1	
1	1	

Fig. 9.2.4 Objective function and constraints for optimization

After this, run the solver by providing the Target Cell, Changing cells and constraints.

Step4: Prediction Function

The optimal solution obtained in Step3 is
u_1=0, u_2=0…. u_7=0, u_8=1.359314, u_9=1.362977….. ,u_{32}=0.669445, u_{33}=0…, u_{48}=0.

Some of the values are zero and while others are not. The values which are not zeros are called as Support vectors. These are the points which lie on the boundary.

As explained earlier the prediction function is given by the following function.

$$f(x) = sign\left(\sum_{i=1}^{48} d_i u_i \phi(x_i)^{\mathrm{T}} \phi(x)\right)$$

Here we get $\sum_{i=1}^{48} d_i u_i \phi(\boldsymbol{x}_i)$ as $\begin{bmatrix} -1.13333 \\ -1.5\mathrm{E}-07 \\ 1.62\mathrm{E}-07 \\ 8.9\mathrm{E}-08 \\ 3.333333 \\ 3.333333 \\ \end{bmatrix}$ (9.1.4)

For predicting the neighbouring points, we take a grid range from -1 to +1, in both X1 and X2 direction. We have taken +1 and -1 as the range because we consider the data samples as normalized ones. We will divide the entire range in 41 equal parts. Hence the delta change between adjacent values will be 0.05.

Note: In case you have used a normalization factor while training then the same logic should be applied while testing.

For calculating the prediction value for each data point, we will again use What-if analysis. At first, each input vector is converted into corresponding feature vector using $\phi(x)$ mapping function. The feature vector is then multiplied with the summation vector as obtained in Eq. 9.1.4. The sign value will tell whether that data point belongs to Class -1 or Class +1.

Step5: Plotting

Here again we will use conditional formatting. In plotting sheet, from cell A1 we will copy the prediction matrix that we got from Step4. After that select the entire table and click the conditional formatting button present in the Home tab, click onto the colour scales and choose the colour you wish.

For circular dataset the plot will be as shown in figure 9.2.5:-

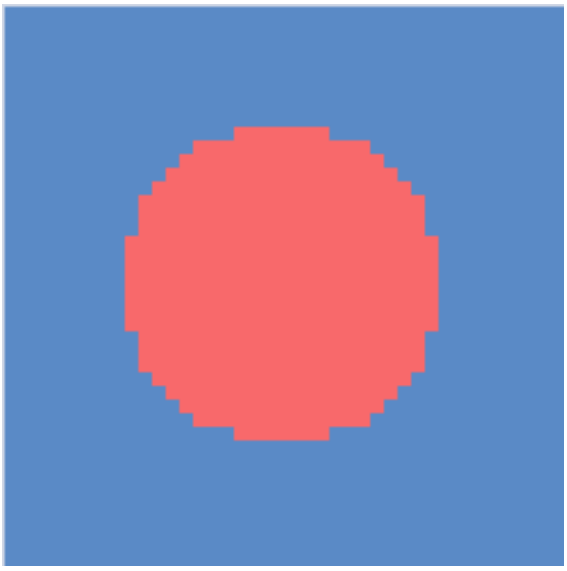


Fig. 9.2.5 Final plot image

Well this again matches to our intution and proves the success of SVM. Red color is given for class -1 and blue for class +1.

9.3 Spiral Problem

9.3.1 Problem Def: This is a little complicated dataset where you have two entangled spirals with starting radius 4 and 7 respectively. Each spiral has 50 sample data points which is generated by the excel formula [2]. The angle between two consecutive spiral points is taken as $\pi / 16$.

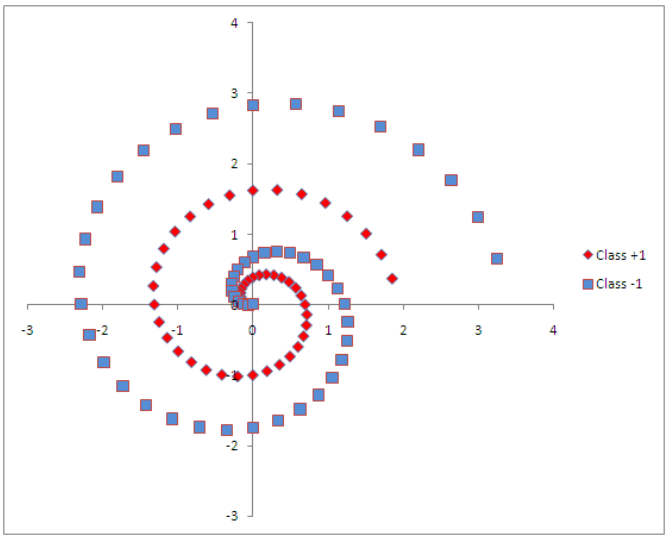


Fig. 9.3.1 XY Plot of input data

	C	D	E	F	G	H
5	Input Data Points					
6	Index	Radius	Theta	X1	X2	Class
7	1	1.884615	0.19635	1.848403	0.36767	1
8	2	1.846154	0.392699	1.705624	0.706492	1
9	3	1.807692	0.589049	1.503041	1.0043	1
10	4	1.769231	0.785398	1.251035	1.251035	1
11	5	1.730769	0.981748	0.961564	1.439082	1
12	6	1.692308	1.178097	0.647618	1.563488	1
13	7	1.653846	1.374447	0.322649	1.622068	1
14	8	1.615385	1.570796	9.9E-17	1.615385	1
56	50	0	9.817477	0	0	1
57	51	3.298077	0.19635	3.234705	0.643423	-1
58	52	3.230769	0.392699	2.984842	1.236362	-1
59	53	3.163462	0.589049	2.630322	1.757525	-1
60	54	3.096154	0.785398	2.189311	2.189311	-1
61	55	3.028846	0.981748	1.682737	2.518394	-1
62	56	2.961538	1.178097	1.133332	2.736105	-1
63	57	2.894231	1.374447	0.564636	2.838619	-1
64	58	2.826923	1.570796	1.73E-16	2.826923	-1
65	59	2.759615	1.767146	-0.53837	2.70659	-1

Fig. 9.3.2 Input Data in excel sheet

9.3.2 Steps for solving this in Excel

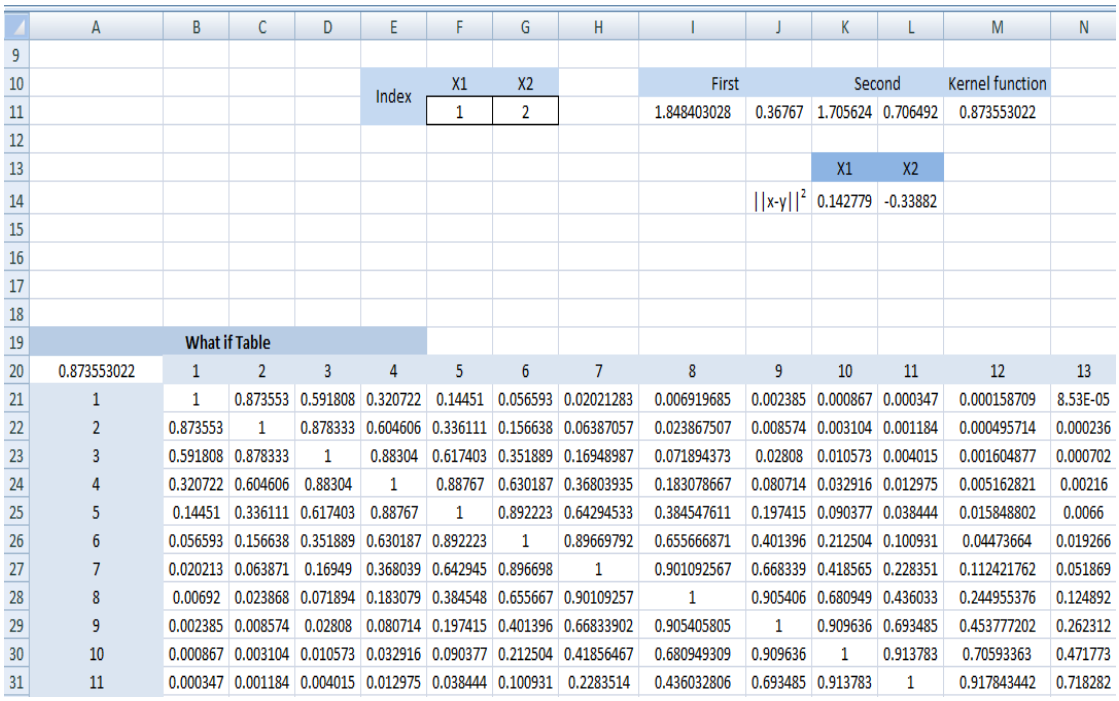
Step1: Understanding the dataset

Figure 9.3.2 shows the columns required for generating spiral data. For the first spiral, the radius column will have the formula =(4*(50-ROW(A1)))/104. This formula will be put for the first data point. The remaining will be automatically generated when you drag this formula down for 50 data points. Here 50-ROW(A1) symbolizes the reduction of the radius as we drag this formula down. 104 is used as the normalizing factor. For the second spiral the formula is =(7*(50-ROW(A1)))/104 which will be written after the 50 data points of spiral 1. The theta column contains the angle. For spiral 1, the first data point will have the formula as =ROW(A1)*(PI()/16).Pull the formula for next 50 rows. After 50 data points, we will again write the same formula for the starting data point of spiral 2 and pull the remaining. One thing you should notice here is that the radius value decreases when we move towards origin. This is the property of any spiral. For generating X1 and X2 cordiantes we will write X1= Radius*COS(Theta) and X2=Radius*SIN(Theta).

Step2: Calculation of K and Q matrices

In Kernel Mapping sheet, you can see that this time we have used a new kernel called exponential or RBF(Radial Basis Function) kernel given as $\mathrm{K}(\boldsymbol{x},\boldsymbol{y}) = exp(-\frac{\sigma}{n}\|\boldsymbol{x}-\boldsymbol{y}\|^2)$. This kernel is used when the problem is highly non-linear and difficult to resolve using polynomial kernels. Here σ is a parameter to control the radius and n is used for nomalizing the kernel. The value of RBF kernel for two vectors x and y basically shows the similarity between them.

RBF is little different from the polynomial kernels as we do not explicitly map the input vector to higher dimension. Since a exponential function is infinite summation of polynomials, we virtually think that this kernel maps a point to infinite dimensional space. For calculating Kernel matrix K, we calculate the similarity measure between all the training data points using the function $\mathrm{K}(\boldsymbol{x},\boldsymbol{y}) = exp(-\frac{\sigma}{n}\|\boldsymbol{x}-\boldsymbol{y}\|^2)$. For spiral case ,we take σ and n as 1. Since this is again a problem of iteration, we will use What-If analysis.



9														
10					Index	X1	X2		First		Second		Kernel function	
11						1	2		1.848403028	0.36767	1.705624	0.706492	0.873553022	
12														
13											X1	X2		
14										$\|\|x-y\|\|^2$	0.142779	-0.33882		
15														
16														
17														
18														
19		What if Table												
20	0.873553022	1	2	3	4	5	6	7	8	9	10	11	12	13
21	1	1	0.873553	0.591808	0.320722	0.14451	0.056593	0.02021283	0.006919685	0.002385	0.000867	0.000347	0.000158709	8.53E-05
22	2	0.873553	1	0.878333	0.604606	0.336111	0.156638	0.06387057	0.023867507	0.008574	0.003104	0.001184	0.000495714	0.000236
23	3	0.591808	0.878333	1	0.88304	0.617403	0.351889	0.16948987	0.071894373	0.02808	0.010573	0.004015	0.001604877	0.000702
24	4	0.320722	0.604606	0.88304	1	0.88767	0.630187	0.36803935	0.183078667	0.080714	0.032916	0.012975	0.005162821	0.00216
25	5	0.14451	0.336111	0.617403	0.88767	1	0.892223	0.64294533	0.384547611	0.197415	0.090377	0.038444	0.015848802	0.0066
26	6	0.056593	0.156638	0.351889	0.630187	0.892223	1	0.89669792	0.655666871	0.401396	0.212504	0.100931	0.04473664	0.019266
27	7	0.020213	0.063871	0.16949	0.368039	0.642945	0.896698	1	0.901092567	0.668339	0.418565	0.228351	0.112421762	0.051869
28	8	0.00692	0.023868	0.071894	0.183079	0.384548	0.655667	0.90109257	1	0.905406	0.680949	0.436033	0.244955376	0.124892
29	9	0.002385	0.008574	0.02808	0.080714	0.197415	0.401396	0.66833902	0.905405805	1	0.909636	0.693485	0.453777202	0.262312
30	10	0.000867	0.003104	0.010573	0.032916	0.090377	0.212504	0.41856467	0.680949309	0.909636	1	0.913783	0.70593363	0.471773
31	11	0.000347	0.001184	0.004015	0.012975	0.038444	0.100931	0.2283514	0.436032806	0.693485	0.913783	1	0.917843442	0.718282

Fig. 9.3.3 (a) Kernel Matrix Calculation

In the figure 9.3.3 (a), the index columns is used by What-If analyser. The first and second columns will have the training vectors as per the index given.(Refer XOR problem incase of difficulty in understanding the index logic). Cell K14 and L14 will have the value $\|x-y\|^2$ which is given by formula =I11:J11-K11:L11. In cell M11, we store the Kernel function value. This is given by formula =EXP(-1*(K14^2+L14^2)). The logic for running the what- if is same. This will give the K matrix.

For matrix Q, we have used the formula $Q = DKD = K.*(d*d^T)$.

Step3: Optimization

This step is exactly similar to the two problems discussed earlier. Once you have formulated the objective function and constraints on the excel sheet, you can run the solver and get the optimum solution.

Step4: Prediction

The optimal solution obtained in Step3 is
u_1=4.02, u_2=0…. u_7=1.95, u_8=2.05, u_9=1.89….. ,u_{60}=0.759091, u_{80}=0022042…, u_{100}=32.39.

As explained earlier the prediction function is given by the following function. $f(x) = sign\left(\sum_{i=1}^{100} d_i u_i \phi(x_i)^T \phi(x)\right)$. But there is twist here. Each value in this summation will be a product of class value d_i , corresponding 'u_i', and the similarity measure between the new test data point 'x' and corresponding training value 'x_i' given by RBF kernel. The sign of the summation will be taken as the class value. For doing this operation, we again take the help of What-If analysis.

Index	$d_i u_i$	$x_i - x$		$\|x_i - x\|^2$	$e^{-1*\|x_i - x\|^2}$	$d_i u_i \phi(x_i)^T \phi(x)$		Class		U vector
1	4.029483607	-2.048403	-0.267670222	4.267602315	0.014015347	0.056474612		1		4.029484
2	0	-1.9056238	-0.606492491	3.999235026	0.018329655	0		1		0
3	0.816175156	-1.7030412	-0.904300037	3.718107961	0.024279863	0.019816621		1		0.816175
4	1.994749234	-1.4510351	-1.151035074	3.43038453	0.032374489	0.064578988		1		1.994749
5	2.046022212	-1.1615639	-1.339082021	3.142371272	0.043180284	0.088347821		1		2.046022
6	1.861540039	-0.8476181	-1.46348844	2.860254884	0.057254165	0.106580921		1		1.86154
7	1.958720114	-0.5226494	-1.522067964	2.589853259	0.075031049	0.146964826		1		1.95872
8	2.05185893	-0.2	-1.515384615	2.336390533	0.096675958	0.198365428		1		2.051859
9	1.891744688	0.10764243	-1.446622942	2.10430483	0.121930407	0.2306612		1		1.891745
97	-19.12836124	-0.0019568	0.060606762	0.003677009	0.996329743	-19.05815524		-1		19.12836
98	-25.04110927	-0.0653846	0.1	0.014275148	0.985826259	-24.68618307		-1		25.04111
99	-31.1949625	-0.1339856	0.113131079	0.030750784	0.969717212	-30.25029207		-1		31.19496
100	-32.39685816	-0.2	0.1	0.05	0.951229425	-30.81684474		-1		32.39686
					Summation	0.454904949				

	x1	x2
Index	-0.2	0.1
Response	1	

=Sign(K112)

Fig. 9.3.3 (b) Prediction function

Figure 9.3.3 (b) shows the screen shot, in the third column x_i-x, x is common for all the rows and the x value is taken from the index cells(shown in blue arrows). Index value are modified by the What-if analyser. Here we take grid range from -4 to +4 in both X1 and X2 direction. We will divide it in 81 equal parts. Hence the delta parameter will be 0.1.

	A	B	C	D	E	F	G	H	I	J	K
115									x1	x2	
116								Index	-0.2	0.1	
117											
118											
119			X1					Response	1		
120	Grid Range -1 to +1		X2								
121											
122											
123											
124	1	-4	-3.9	-3.8	-3.7	-3.6	-3.5	-3.4	-3.3	-3.2	-3.1
125	-4	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
126	-3.9	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
127	-3.8	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
128	-3.7	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
129	-3.6	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
130	-3.5	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Fig. 9.3.4(a) Prediction table calculted using what-if

Figure 9.3.4 shows the prediction table calculated using the whatif analysis. The Row and column input cell for What –if table will link to the index cells.

Step5: Plotting

Here again we will us conditional formatting. In plotting sheet, from cell A1 we will copy the prediction table that we got from Step4. After that select the entire table and click the conditional formatting button present in the Home tab, click onto the colour scales and choose the colour you wish.

For spiral dataset the plot will look as shown in figure 9.3.4(b):-



Fig. 9.3.4(b) Final plot image

You can see how SVM has learned a highly non-linear curve using RBF kernel. The red color is for Class +1 and blue color is for Class -1.

9.4 Support Vector Regression (SVR) problems

9.4.1 Sinc Problem

Let us now focus toward the regression problems. The first problem that we are going to deal is Sinc dataset. Figure 9.4.1 shows a Sinc function.

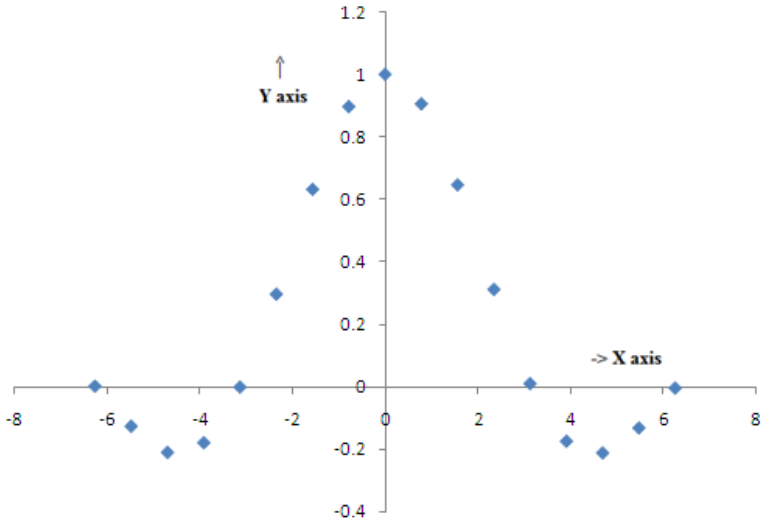


Fig. 9.4.1 XY Plot of input data

The range of the sinc function is from -2*Pi to +2*Pi. The X axis shows the time period and Y axis shows the function value.

9.4.2 Problem Def: The machine has to learn a function from the small set of sinc data points so that it can predict accurately the function value (Y values) at different instances of the time period (X values). In signal processing, this is called as Interpolation.

As this problem is a non-linear one, we will use a Non-linear SVR formulation. We will use Non-linear SV Regression without bias term. The optimization function in dual is given as:-

$$\text{Min } L_D(\mathbf{u}) = \frac{1}{2}\bar{\mathbf{u}}^T \boldsymbol{Q}\bar{\mathbf{u}} - \mathbf{g}^T\bar{\mathbf{u}}$$

$$\text{s.t} \quad \mathbf{0} \le \boldsymbol{u}_i, \boldsymbol{u}_i^* \le C$$

$$\text{where } Q = \begin{bmatrix} K & -K \\ -K & K \end{bmatrix},\ \bar{u} = \begin{bmatrix} u^* \\ u \end{bmatrix},\ g = \begin{bmatrix} \varepsilon - y \\ \varepsilon + y \end{bmatrix}$$

Here L_D is the lagrangian dual function. 'K' is kernel matrix, u and u* are the lagrangian multipliers. ε is the constant which decides the loss function. Lesser the ε value, more the non-linearity is introduced to the prediction function. C is the parameters which decides

whether the optimization function should concentrate more on reducing error or reducing the complexity of the predicted function. Hence ε and C are the crucial factors which decides the generalization of the prediction function. Let us now see how can we implement it in excel.

9.4.3 Steps for solving this in Excel

Step1: Creation of Sinc data in excel

	C	D	E
5	Input Data Points		
6	Index	X	Y
7	1	-6.28319	=SIN(D7)/D7
8	2	-5.5	-0.12828006
9	3	-4.71681	-0.2120054
10	4	-3.93363	-0.18094876
11	5	-3.15044	-0.00280954
12	6	-2.36726	0.29537963
13	7	-1.58407	0.63122821
14	8	-0.80089	0.89647298
15	9	-0.0177	0.99994777
16	10	0.76548	0.90516111
17	11	1.54867	0.6455582
18	12	2.33185	0.31052882
19	13	3.11504	0.00852354
20	14	3.89822	-0.17609959
21	15	4.68141	-0.2135084
22	16	5.46459	-0.13362087
23	17	6.24778	-0.00566574

Fig. 9.4.2 Input Data in excel sheet

Figure 9.4.2 is the snapshot taken from excel. Column X contains data from -2*pi to +2*pi which is from -6.28 till +6.24. The Column Y contain the function value. Sinc function is given as $\frac{\sin(x)}{x}$ where X (cell D7 for first data point) is the data point. You can drag this formula to rest of the data points and then plot the X and Y axis using XY scatter plot. You will get a Sinc function shown in figure 9.4.1.

Step2: Calculation of K and Q matrices.

Calculation of K matrix is exactly similar to the SVM counterpart. We have used the RBF kernel. In this problem we got only one predictor variable 'X'.

	A	B	C	D	E	F	G	H	I	J	K	L
10												
11				Index	X1	X2		First		Second		Kernel function
12					1	2		-6.283185307		-5.5		0.541517863
13												
14									X			
15								$\|\|x-y\|\|^2$	-0.783185307			
16												
17												
18												
19		What if Table										
20	0.541517863	1	2	3	4	5	6	7	8	9	10	11
21	1	1	0.541518	0.085991	0.004004	5.47E-05	2.19E-07	2.5708E-10	8.8517E-14	8.94E-18	2.64628E-22	2.29762E-27
22	2	0.541518	1	0.541518	0.085991	0.004004	5.47E-05	2.1894E-07	2.57075E-10	8.85E-14	8.93755E-18	2.64628E-22
23	3	0.085991	0.541518	1	0.541518	0.085991	0.004004	5.4677E-05	2.18937E-07	2.57E-10	8.8517E-14	8.93755E-18
24	4	0.004004	0.085991	0.541518	1	0.541518	0.085991	0.00400419	5.4677E-05	2.19E-07	2.57075E-10	8.8517E-14
25	5	5.47E-05	0.004004	0.085991	0.541518	1	0.541518	0.08599063	0.004004194	5.47E-05	2.18937E-07	2.57075E-10
26	6	2.19E-07	5.47E-05	0.004004	0.085991	0.541518	1	0.54151786	0.085990633	0.004004	5.4677E-05	2.18937E-07
27	7	2.57E-10	2.19E-07	5.47E-05	0.004004	0.085991	0.541518	1	0.541517863	0.085991	0.004004194	5.4677E-05
28	8	8.85E-14	2.57E-10	2.19E-07	5.47E-05	0.004004	0.085991	0.54151786	1	0.541518	0.085990633	0.004004194

Fig. 9.4.3 Kernel Matrix Calculation

Matrix Q is calculated using the formula :-

$$Q = \begin{bmatrix} K & -K \\ -K & K \end{bmatrix}$$

	C	D	E	F	G	H	I	J
50	Q Matrix							
51	=C21	0.085991	0.004004	5.47E-05	2.19E-07	2.5708E-10	8.8517E-14	8.94E-18
52	1	0.541518	0.085991	0.004004	5.47E-05	2.1894E-07	2.57075E-10	8.85E-14
53	0.541518	1	0.541518	0.085991	0.004004	5.4677E-05	2.18937E-07	2.57E-10
54	0.085991	0.541518	1	0.541518	0.085991	0.00400419	5.4677E-05	2.19E-07
55	0.004004	0.085991	0.541518	1	0.541518	0.08599063	0.004004194	5.47E-05
56	5.47E-05	0.004004	0.085991	0.541518	1	0.54151786	0.085990633	0.004004
57	2.19E-07	5.47E-05	0.004004	0.085991	0.541518	1	0.541517863	0.085991
58	2.57E-10	2.19E-07	5.47E-05	0.004004	0.085991	0.54151786	1	0.541518
59	8.85E-14	2.57E-10	2.19E-07	5.47E-05	0.004004	0.08599063	0.541517863	1
60	8.94E-18	8.85E-14	2.57E-10	2.19E-07	5.47E-05	0.00400419	0.085990633	0.541518
61	2.65E-22	8.94E-18	8.85E-14	2.57E-10	2.19E-07	5.4677E-05	0.004004194	0.085991
62	2.3E-27	2.65E-22	8.94E-18	8.85E-14	2.57E-10	2.1894E-07	5.4677E-05	0.004004
63	5.85E-33	2.3E-27	2.65E-22	8.94E-18	8.85E-14	2.5708E-10	2.18937E-07	5.47E-05

Fig. 9.4.4 Q matrix calculation

In the figure 9.4.4 the cell C21 refers to the first cell of matrix K. You will have to drag the formula (Both row and column wise) till the full matrix is printed.

Step3: Optimization

You will be formulating the problem as:-

$$\text{Min } L_D(\mathbf{u}) = \frac{1}{2}\bar{\mathbf{u}}^T \mathbf{Q}\bar{\mathbf{u}} - \mathbf{g}^T\bar{\mathbf{u}}$$

$$\text{s.t} \quad \mathbf{0} \le \boldsymbol{u}_i, \boldsymbol{u}_i^* \le C$$

$$\text{where } Q = \begin{bmatrix} K & -K \\ -K & K \end{bmatrix}, \ \bar{u} = \begin{bmatrix} u^* \\ u \end{bmatrix}, \ g = \begin{bmatrix} \varepsilon - y \\ \varepsilon + y \end{bmatrix}$$

In optimization sheet, at cell L17 we have written the objective function. The formula used is =0.5*MMULT(MMULT(TRANSPOSE(J20:J53),'Kernel Mapping'!B51:AI84),Optimization!J20:J53)+MMULT(TRANSPOSE(Optimization!I20:I53),Optimization!J20:J53).

Range B51:AI84 of Kernel Mapping sheet contains the Q matrix. Range J20:J53 of Optimization sheet contains the $\bar{u}$ (Dimension of $\bar{u}$ is 2n x 1) vector and Optimization!I20:I53 contains the vector g. Vector 'g' is in Column 'I'. The first cell value of g contains the formula =$E$17-D20. You can drag it till 'n' input data points. After that change the formula to =$E$17+D20 and again drag it till the end. Hence the dimension of g is also 2n x 1. ε is fixed as 0.05 and is put in cell E17.

The vector 'y' is copied from the Input sheet to Optimization sheet in column D.

=0.5*MMULT(MMULT(TRANSPOSE(J20:J53),'Kernel Mapping'!B51:AI84),Optimization!J20:J53)+MMULT(TRANSPOSE(Optimization!I20:I53),Optimization!J20:J53)

	D	E	F	G	H	I	J	K	L	M	N
16											
17	epsilon	0.05					Objective Function		-0.77992		
18											
19	y					g	uBar		u*	u	u*-u
20	-3.89977E-17					0.05	0		0	0	0
21	-0.128280059					0.17828	0		0	0.007396	-0.0074
22	-0.212005404					0.262005	0		0	0.119603	-0.1196
23	-0.180948756					0.230949	0		0	0.071437	-0.07144
24	-0.002809543					0.05281	0		0	0	0
25	0.295379635					-0.24538	0.054166		0.054166	0	0.054166
26	0.631228212					-0.58123	0.30066		0.30066	0	0.30066
27	0.89647298					-0.84647	0.386601		0.386601	0	0.386601
28	0.999947769					-0.94995	0.475376		0.475376	0	0.475376
29	0.905161114					-0.85516	0.392274		0.392274	0	0.392274
30	0.645558199					-0.59556	0.30544		0.30544	0	0.30544
31	0.310528824					-0.26053	0.065493		0.065493	0	0.065493
32	0.00852354					0.041476	0		0	0	0

Fig. 9.4.5 Objective function and constraints for optimization

We have split the $\bar{u}$ vector into u and u* vector. They are put in columns L and M respectively. In column N the differnce of u* and u is given. The vector $\bar{u}$ is initialized to 1's.

Now we can run the solver with all necessary parameters.

Note: We have given the **C** value as 1 in the constraints (See constraint in figure 9.4.6). You can change it and see how the prediction results vary due to it.

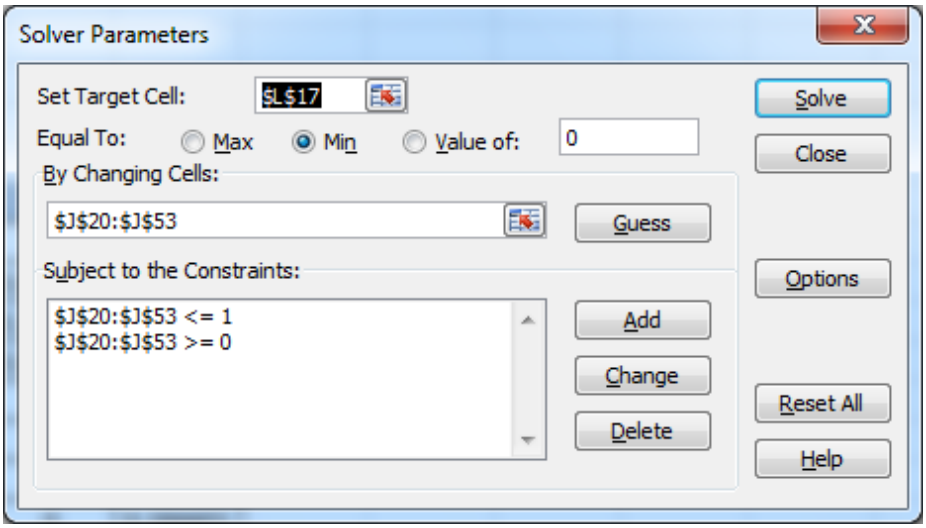


Fig. 9.4.6 Solver parameters

Solver will find the optimum solution in couple of seconds.

Step4: Prediction

The prediction function used in SVR is:-

$$f(x) = \sum_{i=1}^{m} \left(u_i^* - u_i\right) \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x})$$

Here $\phi(\boldsymbol{x})$ belongs to the new data point. The prediction is similar to one explained for Spiral dataset. Figure 9.4.7 shows how we did it in excel.

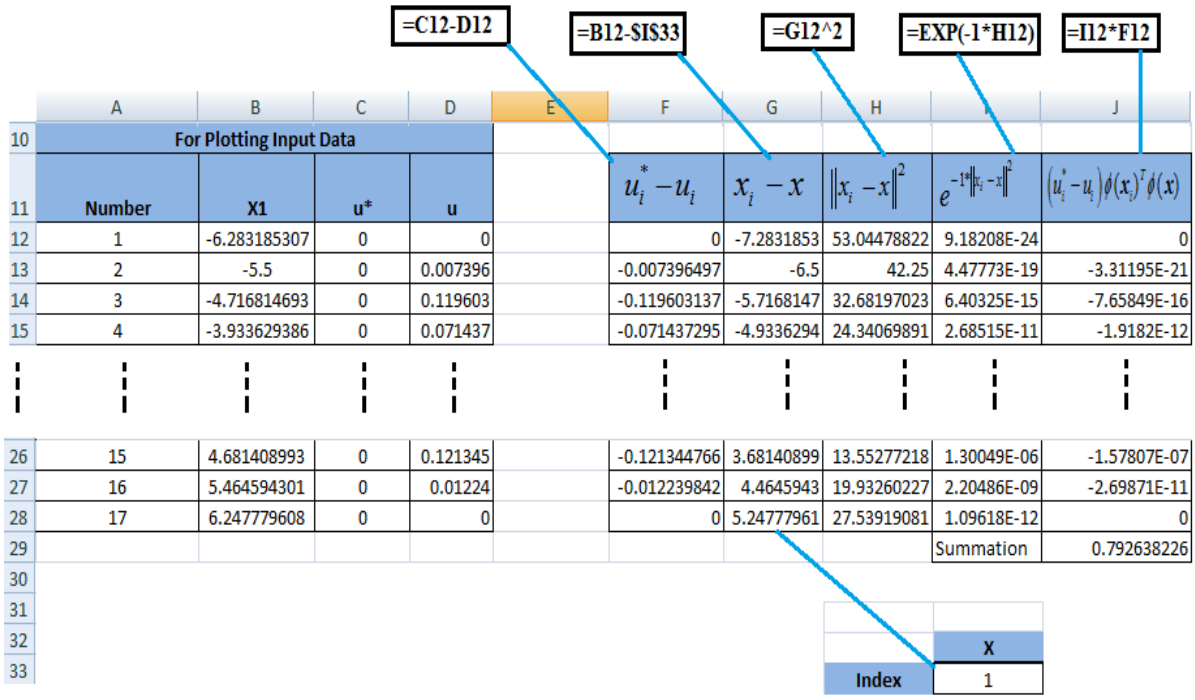


Fig. 9.4.7 Inputs required for prediction function

Each time when the index changes, the summation value will change.The summation value is the predicted value for the current X. Inorder make all prediction in one stretch, we will again take help of what-if analysis. In the first column, we will write all the X value for which we want the prediction. Here in this case we have taken the values from -2*pi to +2*pi with the delta value as 0.1 (See column A)

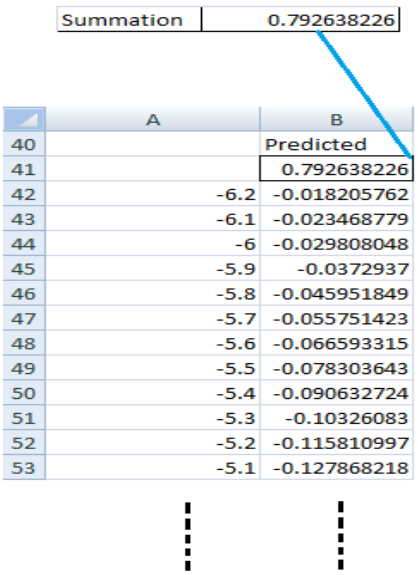


Fig. 9.4.8 Prediction table calculted using what-if

The first value in column B i.e. cell B41 will be linked to the summation value. After this we will run what-if analysis and select the column input cell as the index cell. This will give prediction for all the test data.

Step5: Plotting

The prediction values that we have generated in Step4 will be plotted using the XY scatter plot. The X axis will vary from -2*pi to +2*pi and Y axis will be the prediction values.

Figure 9.4.9 shows the output that we got.

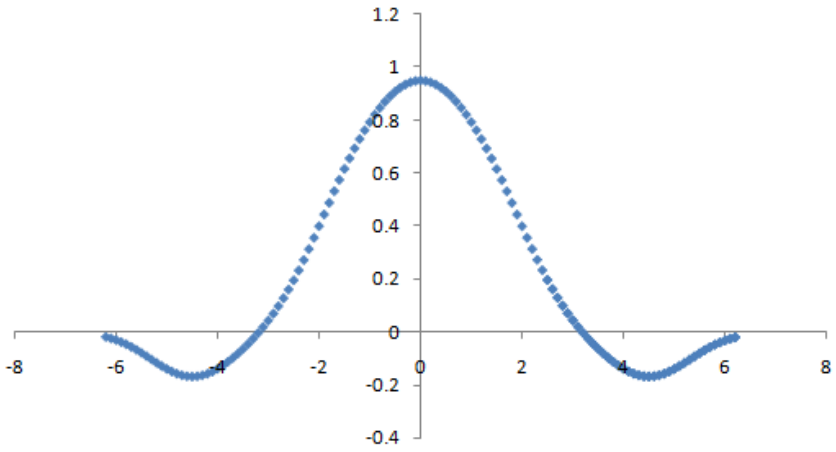


Fig. 9.4.9 Plotting the learned function

This is exactly what we wanted. SVR has learned the Sinc signal and interpolated all the values properly.

9.5 Eigen Ear Problem

9.5.1 Problem def: This is a real world pattern recognition problem where we have to identify a person based on its ear pattern. Currently, we are training the machine to recognize between two persons. For more info about the problem and the preprocessing required for getting the training feature vectors, refer the Neural Network documentation for the same.

We have solved this problem exactly as the Spiral problem where we used the RBF kernel.

9.5.2 Steps for solving this in Excel

Step1: The feature vectors that we get after preprocessing is stored in the Training Data sheet of the excel file. Here we are taking only two persons data.

TRAINING DATA						
	Ist persons data					
1	-58.7711	280.9096	-69.4127	-58.1208	12.58001	-45.3947
2	174.2977	-81.0289	210.2409	-43.7435	8.312412	-72.1541
3	-152.236	-241.018	11.5745	-58.045	27.76317	-48.3861
4	249.826	-125.807	-87.4912	-56.6667	-11.6741	-6.15202
5	218.4901	-116.428	-157.963	-6.4067	47.61373	-26.5181
6	248.2901	-120.408	-95.674	-28.6108	16.93376	-25.2081

	2nd persons data					
1	30.54459	18.7899	-264.358	66.81264	-21.2255	-30.2026
2	-210.867	-165.86	-48.0033	57.71137	-49.5129	-14.3416
3	56.07401	-5.60453	-237.789	81.91401	-59.2399	62.4874
4	26.46499	219.6633	-46.32	122.6258	-105.002	-46.336
5	-90.8103	79.14017	-207.479	-76.3416	-87.2376	-55.6533
6	112.1995	50.47063	-237.522	59.35626	13.12117	-59.0739

TESTING DATA						
	Ist persons data					
1	75.20759	-117.541	-116.063	-48.5474	81.82071	-220.396
2	109.9676	-68.2546	-18.3594	-168.383	215.0814	-2.57751
3	-194.46	-164.618	-78.2745	70.27048	68.69451	-1.19464
4	199.4986	-5.19013	15.66583	170.7978	142.45	-55.3007

	2nd persons data					
1	91.63448	-41.0071	-225.52	14.40203	-149.799	-12.3858
2	145.3565	-7.72998	-164.817	44.65883	-148.245	-13.8282
3	278.1985	-25.1246	-12.5512	-28.1766	-21.2124	-4.24167
4	-64.5743	226.0808	-141.025	-15.2914	-44.8668	48.78289

Fig. 9.5.1 Input Data in excel sheet

Step2: In input sheet, we have copied the all the training vectors along with their class values.

	K	L	M	N	O	P	Q	R
5		Normalized Data Points						
6	Index	X1	X2	X3	X4	X5	X6	Class
7	1	-0.69803	1.907355	0.248541	-0.92471	0.622773	-0.42262	1
8	2	0.793667	-0.40788	2.348168	-0.71422	0.533817	-1.18586	1
9	3	-1.29622	-1.4313	0.856589	-0.9236	0.939256	-0.50794	1
10	4	1.277065	-0.69432	0.112809	-0.90342	0.117212	0.696685	1
11	5	1.076508	-0.63432	-0.41629	-0.16759	1.353028	0.115792	1
12	6	1.267235	-0.65978	0.051373	-0.49267	0.713524	0.153157	1
13	7	-0.12639	0.230636	-1.2151	0.904376	-0.08188	0.0107	-1
14	8	-1.67148	-0.95052	0.409282	0.771129	-0.67151	0.463096	-1
15	9	0.037007	0.07459	-1.01562	1.125468	-0.87427	2.654457	-1
16	10	-0.1525	1.515576	0.42192	1.721508	-1.82814	-0.44947	-1
17	11	-0.90309	0.616682	-0.78806	-1.19147	-1.45786	-0.71522	-1
18	12	0.396223	0.43329	-1.01362	0.795211	0.634053	-0.81278	-1

Fig. 9.5.2 Normalized input data

For normalizing the data points, we use the formula $(x_i - \mu_i)/\sigma_i$ where x_i is current data point, μ_i is the mean value of that predictor variable, σ_i is the standard deviation for that predictor variable. Here X1,X2…X6 are the predictor variables. Hence the mean of X1 predictor variable will be given by excel formula =AVERAGE(B7:B18) where B7:B18 contains the values of X1for different input data. Similarly the standard deviation is calculated using the excel formula =STDEV(B7:B18).

Step3: Calculation of K and Q matrices and running the solver.

These two steps are same as we did for the spiral problem.

Step4: Prediction

In this step, when we predict for a new data point, it has to be passed through the same normalization process. That means we have to use the same value for mean and standard deviation as we used for the original input data.

Finding out the summation value is exactly same as we did in spiral.

		X1	X2	X3	X4	X5	X6
Index	2	0.793666531	-0.40788195	2.348168	-0.71422	0.533817	-1.1858639
	Response	1					

	Normalized Testing Data						
S.No	X1	X2	X3	X4	X5	X6	Actual Class
1	-0.69803	1.907355	0.248541	-0.9247113	0.622773	-0.42262	1
2	0.793667	-0.40788	2.348168	-0.7142211	0.533817	-1.18586	1
3	-1.29622	-1.4313	0.856589	-0.9236016	0.939256	-0.50794	1
4	1.277065	-0.69432	0.112809	-0.9034225	0.117212	0.696685	1
5	1.076508	-0.63432	-0.41629	-0.1675916	1.353028	0.115792	1
6	1.267235	-0.65978	0.051373	-0.4926701	0.713524	0.153157	1
7	-0.12639	0.230636	-1.2151	0.90437614	-0.08188	0.0107	-1

	Prediction Results	
Sno	1	True/False
1		FALSE
2		FALSE
3		FALSE
4		FALSE
5		FALSE
6		FALSE
7		FALSE

Fig. 9.5.3 Prediction function

For testing, we gave both the training data and the 4 test data for each person.

In figure 9.5.3, on the top you can see the index cell. Based on the index value, the testing data will populate on the indexes X1 to X6. This new vector is later used for calculating the summation value. The response will depend on the sign of the summation value. The Prediction results table is created by the what-if analyser. In the first column, we have put the row index

of the test data (Sno) and the first cell of the table i.e. T42 is linked to the response cell. Now run the What-if analyser and select the column input cell as index cell. You can see the prediction done for each each input test data.

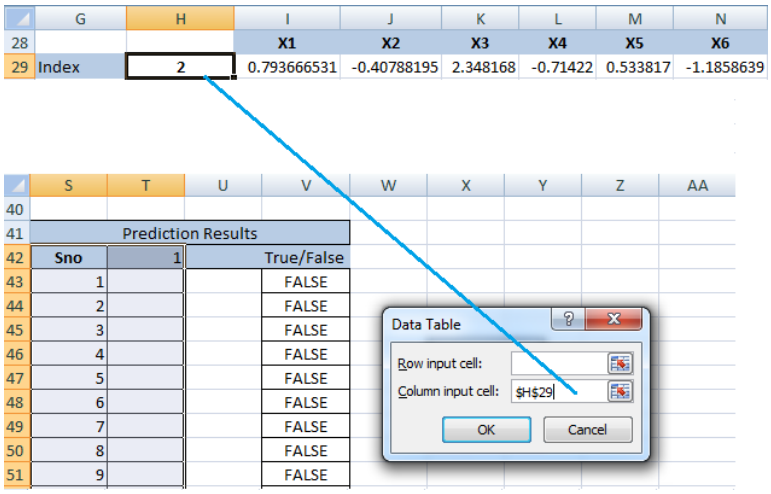


Fig. 9.5.4 Running What-if analysis

You can see that SVM has predicted all the training vectors properly. Out of four new test input for each person, 3 are correct and 1 is misclassified. This brings an accuracy of 75% to our algorithm. For the misclassified data ,there could be a possibilty that it came due to some error in preprocessing or because of inadequate features.

Note: The Breast Cancer Reccurance Predictor problem is very similar to Eigen Ear problem. You can refer the cd for SVM_Cancer.xlsx workbook for understanding the same.

9.6 Time Series Analysis

9.6.1 Problem def: This is again a regression problem which we have broadly discussed in Neural network section and have seen many approaches to solve them. In SVR approach the window size is taken as 3. i.e. the new sales value for a Quarter will depend on the past three quarters sale. Figure 9.6.1 shows the training data.

9.6.2 Steps for solving this in excel:

Input Data Points				
INDEX	T1	T2	T3	NEW VALUE
1	642.6136364	650.7172131	655.6985294	647.6123596
2	650.7172131	655.6985294	647.6123596	729.1193182
3	655.6985294	647.6123596	729.1193182	722.0286885
4	647.6123596	729.1193182	722.0286885	719.6691176
5	729.1193182	722.0286885	719.6691176	733.1460674
6	722.0286885	719.6691176	733.1460674	803.2670455
7	719.6691176	733.1460674	803.2670455	802.2540984
8	733.1460674	803.2670455	802.2540984	799.6323529
9	803.2670455	802.2540984	799.6323529	794.241573
Mean	700.4302195	718.1680486	734.7141753	750.1078468
SD	54.66971699	59.27045367	58.87140894	53.49543791

Normalized Data Points				
INDEX	T1	T2	T3	NEW VALUE
1	-1.057561413	-1.138017871	-1.342173515	-1.91596688
2	-0.909333524	-1.053974035	-1.479526604	-0.3923424
3	-0.818216969	-1.190402379	-0.095035217	-0.52488884
4	-0.966126457	0.18476777	-0.215477886	-0.56899673
5	0.524771304	0.065135994	-0.255557968	-0.31706964
6	0.395071901	0.025325756	-0.026636153	0.99371462
7	0.351911427	0.252706329	1.164450986	0.97477941
8	0.598427241	1.435774346	1.14724489	0.92577065
9	1.88105649	1.418684092	1.102711466	0.82499981

Fig. 9.6.1 Normalized input data in excel sheet

You can see that the normalization is done through mean and standard deviation of each predictor variable.

All the remaining steps are same as we have done for sinc dataset. The error obtained during prediction varies when we change the ε and C parameter in the constraint. Table 9.6.1 shows some of the statistics that we got on different values.

ε	C	Error
0.05	0.1	323.282188
0.05	1	155.4778892
0.05	10	114.0073282
0.05	100	24.07660231
0.1	1	169.2528337
1	1	321.6658471
10	1	397.9273653

Table 9.6.1 Variation of Error w.r.t ε and C

You can clearly see that when we increase the C value, the error decreases due to high non-linear prediction function. While on increasing ε value , the error decreases initially but later on stays constant. In the real world applications, these parameters should be tuned according to our needs and they should bring less generalisation error.

9.7 Multi-class Support Vector Machines

Most of the real world problems are Multi-class i.e. they have more than two output classes to predict. In this section we are going to deal a problem where we have three output classes to predict. As you saw in chapter 8, there are many approaches to tackle a multi-class problem. In the excel implementation, we have used a direct method which is based on proximal support vector machines. Here the output will be a vector, based on which we will predict its class. You have already seen the formulation in chapter 8. Our first goal will be to find out the lagrangian multipliers.

Here $\alpha = \mathbf{Q}^{-1}\mathbf{e}$ where $\mathbf{Q} = (\mathbf{K} + \frac{\mathbf{I}}{C})$

$\mathbf{K}$ is given as:-

$\mathbf{K} = \mathbf{K}^{y} .* \mathbf{K}^{\phi}$ (.* represents elementwise multiplication)

$\mathbf{K}^{y}$ is the kernel matrix of output labels.

$\mathbf{K}^{\phi}$ is the kernel matrix of feature vectors.

Using α, we can find the W matrix which will help for the prediction.

The prediction function predicting the labels is expressed by using the predicted vector output

$$d(\mathbf{x}) = \arg \max_{t=1,\ldots,T} \left\langle \hat{\mathbf{y}}_t, \mathbf{W}\phi(\mathbf{x}) \right\rangle \quad (9.7.1)$$

$$= \arg \max_{t = 1,\ldots, T} \sum_{i=1}^{m} \alpha_i K^{y}(\hat{\mathbf{y}}_t, \mathbf{y}_i) K^{\phi}(\mathbf{x}_i, \mathbf{x})$$

9.7.1 Problem Def: This problem is about choosing the best Contact Lens for a patient. We have explained the problem in the neural network section. Please refer to understand the pre-processing done while selecting the feature vectors.

9.7.2 Steps done in excel:

Step1: Understanding the data and normalizing it based on the mean and standard deviation of each predictor variable. Figure 9.7.1 shows the normalized data from the input s`heet.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
6	Generated Data Points/Training Data											Normalized Generated Data Points								
7						Class values											Class values			
8	index	X1	X2	X3	X4	Integer	Binary			=(B9-BS33)/BS34		index	X1	X2	X3	X4	Integer	Binary		
9	1	0	0	0	0	1	1	0	0			1	-1.19896	-0.97895	-0.97895	-0.97895	1	1	0	0
10	2	0	0	0	1	3	0	0	1			2	-1.19896	-0.97895	-0.97895	0.978945	3	0	0	1
11	3	0	0	1	0	1	1	0	0			3	-1.19896	-0.97895	0.978945	-0.97895	1	1	0	0
12	4	0	0	1	1	2	0	1	0			4	-1.19896	-0.97895	0.978945	0.978945	2	0	1	0
13	5	0	1	0	0	1	1	0	0			5	-1.19896	0.978945	-0.97895	-0.97895	1	1	0	0
14	6	0	1	0	1	3	0	0	1			6	-1.19896	0.978945	-0.97895	0.978945	3	0	0	1
	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮			⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
29	21	2	1	0	0	1	1	0	0			21	1.198958	0.978945	-0.97895	-0.97895	1	1	0	0
30	22	2	1	0	1	3	0	0	1			22	1.198958	0.978945	-0.97895	0.978945	3	0	0	1
31	23	2	1	1	0	1	1	0	0			23	1.198958	0.978945	0.978945	-0.97895	1	1	0	0
32	24	2	1	1	1	1	1	0	0			24	1.198958	0.978945	0.978945	0.978945	1	1	0	0
33	Mean	1	0.5	0.5	0.5															
34	SD	0.834058	0.510754	0.510754	0.510754															
35																				

Fig. 9.7.1 Normalized input data in excel

We have four predictor variables from X1 to X4.You can notice that the Class value in integer column vary from 1 to 3. In the binary column we have represented each class with a 3 bit unique binary code. For e.g. for class 1 the binary code is 1 0 0. For class 2 it will 0 1 0. For class 3 it will be 0 0 1. For creating binary code we have used simple excel formula =IF(F9=1,1,0). Here F9 is the column which has integer values.

The following are the rules used for setting up label matrix.

$$(y_i)_t = \begin{cases} \sqrt{\dfrac{T-1}{T}} & \text{if item } i \text{ belongs to category } t,\ t = 1, \ldots, T, \\ -\dfrac{1}{\sqrt{T(T-1)}} & \text{otherwise,} \end{cases}$$

Hence for 3-class classification, it is given as

$$y_1 = \begin{bmatrix} \sqrt{2/3} \\ -1/\sqrt{6} \\ -1/\sqrt{6} \end{bmatrix}, \; y_2 = \begin{bmatrix} -1/\sqrt{6} \\ \sqrt{2/3} \\ -1/\sqrt{6} \end{bmatrix}, \; y_3 = \begin{bmatrix} -1/\sqrt{6} \\ -1/\sqrt{6} \\ \sqrt{2/3} \end{bmatrix} \qquad (9.7.2)$$

You can see this matrix (Figure 9.7.2) present in the lower part of the input sheet.

	H	I	J	K
45	**Label Matrix**			
46	**y1**	0.8165	-0.4082	-0.4082
47	**y2**	-0.4082	0.8165	-0.4082
48	**y3**	-0.4082	-0.4082	0.8165

Fig. 9.7.2 Label Matrix

Step2: Finding the kernel matrix for feature vectors ($\mathbf{K}^{\phi}$)

Note: All calculations are done in the Calculation sheet.

Here we will use what-if analysis (Figure 9.7.3).

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
6	**Kernel Matrix of Feature Vectors**															
7																
8																
9	**index**	**Data1**	**Data2**		**X1**	**X2**	**X3**	**X4**		**X1**	**X2**	**X3**	**X4**		**Kernel Function**	
10		20	20		1.198958	-0.97895	0.978945	0.978945		1.198958	-0.97895	0.97894501	0.978945		1	
11																
12																
13	1	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
14	1	1	0.147096	0.14709647	0.021637	0.147096	0.021637	0.021637	0.003183	0.487361	0.071689	0.071689093	0.010545	0.071689	0.010545212	0.010545
15	2	0.147096	1	0.02163737	0.147096	0.021637	0.147096	0.003183	0.021637	0.071689	0.487361	0.010545212	0.071689	0.010545	0.071689093	0.001551
16	3	0.147096	0.021637	1	0.147096	0.021637	0.003183	0.147096	0.021637	0.071689	0.010545	0.487361077	0.071689	0.010545	0.001551163	0.071689
17	4	0.021637	0.147096	0.14709647	1	0.003183	0.021637	0.021637	0.147096	0.010545	0.071689	0.071689093	0.487361	0.001551	0.010545212	0.010545
18	5	0.147096	0.021637	0.02163737	0.003183	1	0.147096	0.147096	0.021637	0.071689	0.010545	0.010545212	0.001551	0.487361	0.071689093	0.071689

Fig. 9.7.3 Kernel matrix for feature vector

Here Data1 and Data2 cells contain the index of the two input vectors for which we calculate the kernel function value (Shown in O10 cell).Here also we will use the RBF kernel with σ as 0.5

The logic of fetching the input vectors is same as explained in the previous problems. In this problem we have 24 set of data. The first row (B13:Y13) and first column (A14:A37) of what-if will have the index value of the data. The first cell in this table i.e. A13 will refer to the Kernel function value which is calculated in the O10 cell. Now you can run the what-if analyzer and select the row input cell as B10 and column input cell as C10. This will give your $\mathbf{K}^{\phi}$ matrix.

Step3: Calculating the kernel for output labels

This is very similar to step2 except the input data vectors will change to input labels.

	A	B	C	D	E	F	G	H	I	J	K	L
42	Kernel Matrix of Label Vectors											
43												
44												
45	index	1st	2nd		b1	b2	b3		b1	b2	b3	Kernel Function
46		1	1		1	0	0		1	0	0	1
47												
48												
49	1	1	2	3	4	5	6	7	8	9	10	11
50	1	1	0.367879	1	0.367879	1	0.367879	1	0.367879	1	0.367879	1
51	2	0.367879	1	0.36787944	0.367879	0.367879	1	0.367879	0.367879	0.367879	1	0.367879441
52	3	1	0.367879	1	0.367879	1	0.367879	1	0.367879	1	0.367879	1
53	4	0.367879	0.367879	0.36787944	1	0.367879	0.367879	0.367879	1	0.367879	0.367879	0.367879441
54	5	1	0.367879	1	0.367879	1	0.367879	1	0.367879	1	0.367879	1

Fig. 9.7.4 Kernel matrix for label vectors

The 1st and 2nd cell contains the index of the output labels. The b1, b2, b3 cells contain the 3 bit binary code of the input data at respective index. The binary labels are fetched from the input sheet. We use the same RBF kernel parameter as we took in step2. You can create the what-if table in a similar way as done in step2. This will give you the kernel matrix for output labels.

Step4: Calculation of $\frac{\mathbf{I}}{C}$ matrix where I represents an identity matrix and 'C' is a tuning parameter

Since we have 24 input data, we require a 24x24 identity matrix. We have used a simple logic for creating an identity matrix using what-if analysis. In figure9.7.5 you can see the cell value (D81) contains a formula which returns 1 if Row and column cells are equal and return 0 otherwise. This means all the diagonal elements of the what-if table will be 1's and rest will 0's. Now you can create a what-if table with the first row and column containing the index values starting from 1 till 24.

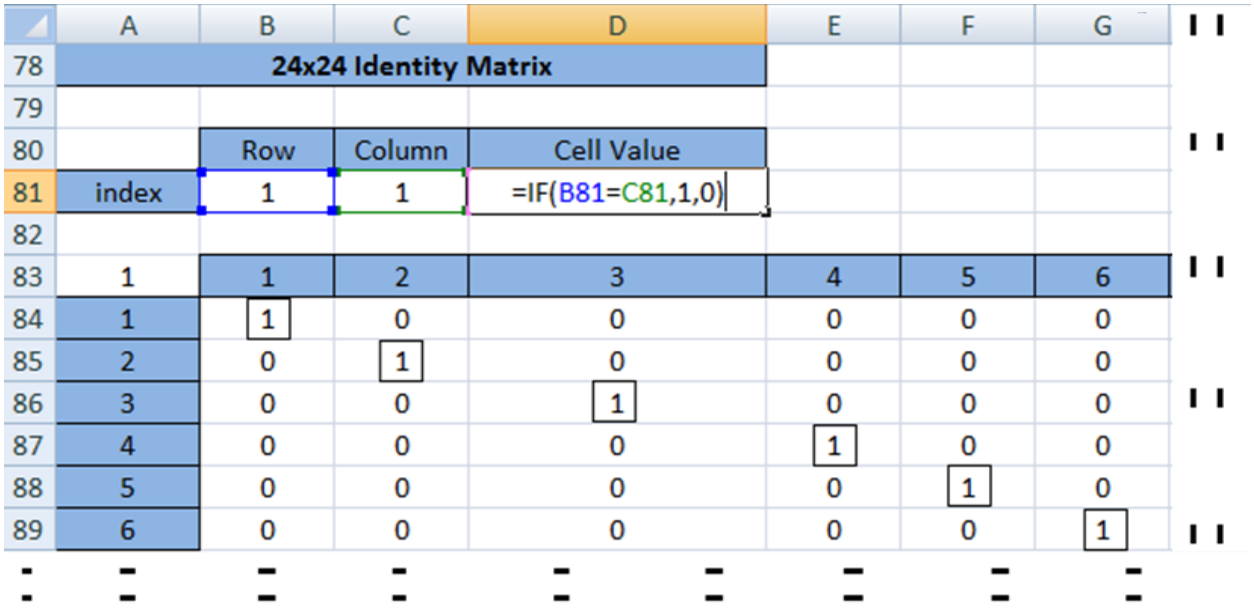


Fig. 9.7.5 Identity matrix using what-if

Parameter C is taken as 0.1 (cell B111). For getting the $\frac{\mathbf{I}}{C}$ matrix, we have written the formula =B84:Y107/$B$111. In this B84:Y107 is the range of the entire identity matrix.

Step5: Calculation of matrix K which is given as $\mathbf{K} = \mathbf{K}^{y} .* \mathbf{K}^{\phi}$

This is simple. We just have to do an element by element multiplication (not matrix multiplication) of the matrices that we get in step2 and 3. In figure 9.7.6, you can see that the first cell contains the formula B14*B50. Here B14 refers to the first cell of $\mathbf{K}^{\phi}$ matrix while B50 refers to the first cell of $\mathbf{K}^{y}$. You can drag this formula to the next 24 columns and rows to populate the entire matrix.

	A	B	C	D	E	F
142	$\mathbf{K} = \mathbf{K}^{y} .* \mathbf{K}^{\phi}$ (.* represents elementwise multiplication)					
143						
144						
145		=B14*B50	0.054114	0.147096467	0.00796	0.147096
146		0.054114	1	0.007959944	0.054114	0.00796
147		0.147096	0.00796	1	0.054114	0.021637
148		0.00796	0.054114	0.054113766	1	0.001171
149		0.147096	0.00796	0.021637371	0.001171	1

Fig. 9.7.6 Element-element multiplication of K matrices

Step6: Calculating $(\mathbf{K} + \frac{\mathbf{I}}{C})$ matrix

This is also similar to step5. Figure 9.7.7 explains it.

	A	B	C	D	E	F	G
174	$(\mathbf{K} + \frac{\mathbf{I}}{C})$						
175							
176							
177							
178		=B145+B114	0.054114	0.147096467	0.00796	0.147096	0.00796
179		0.05411377	11	0.007959944	0.054114	0.00796	0.147096
180		0.14709647	0.00796	11	0.054114	0.021637	0.001171
181		0.00795994	0.054114	0.054113766	11	0.001171	0.00796
182		0.14709647	0.00796	0.021637371	0.001171	11	0.054114

Fig. 9.7.7 Calculating $(\mathbf{K} + \frac{\mathbf{I}}{C})$ matrix

Step7: Calculating $\alpha = \mathbf{Q}^{-1}\mathbf{e}$

α is the vector of lagrangian multipliers and Q is given as $\mathbf{Q} = (\mathbf{K} + \frac{\mathbf{I}}{C})$. '**e**' represents the column vector of all 1's. In our case the dimension of column vector will be 24 x 1.

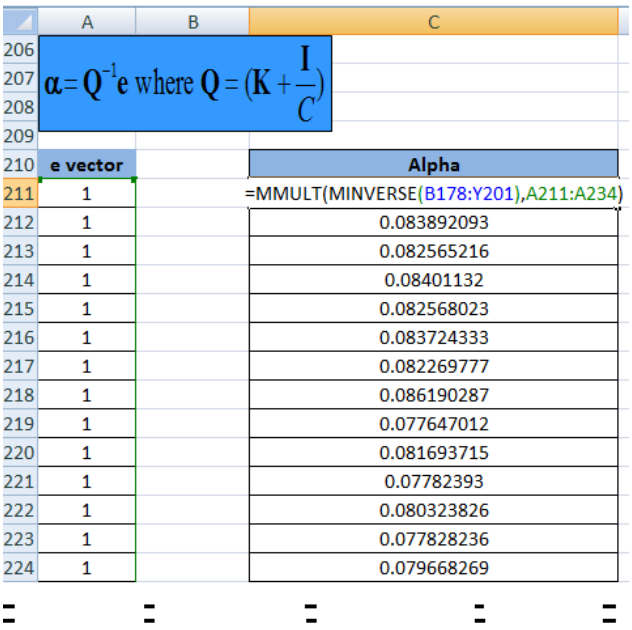


Fig. 9.7.8 Calculating $\alpha = \mathbf{Q}^{-1}\mathbf{e}$

In excel 'e' vector is put in cell range A211:A234. For calculating α, we have used the formula =MMULT(MINVERSE(B178:Y201),A211:A234). Here range B178:Y201 refers to matrix Q which we found out in step6. Once you have calculated the α, you can move on to the Prediction steps.

Step8: Prediction

The decision function predicting the labels is expressed by using the predicted vector output

$$d(x) = \arg \max_{t=1,\ldots,T} \sum_{i=1}^{m} \alpha_i K^{y}(\hat{\mathbf{y}}_t, \mathbf{y}_i) K^{\phi}(\mathbf{x}_i, \mathbf{x}) \tag{9.7.3}$$

Here T is the no of classes. α is calculated from step7. 'x' is the new data point for which we have to predict. $K^{\phi}(\mathbf{x}_i, \mathbf{x})$ refers to feature kernel vector with respect to the new vector x. $K^{y}(\hat{\mathbf{y}}_t, \mathbf{y}_i)$ refers to the output label kernel vector with respect to the output label 't'. In excel we did the calculations in the following order.

Step8.1 Calculating $K^{y}(\hat{\mathbf{y}}_t, \mathbf{y}_i)$

This is done with the help of What-if analysis.

	A	B	C	D	E	F	G	H	I	J	K	L
6	index	Label	Training Data Label		I1	I2	I3		b1	b2	b3	Kernel Function
7		3	1		-0.4082	-0.4082	0.8165		1	0	0	=EXP(-0.5*((I7-E7)^2+(J7-F7)^2+(K7-G7)^2))

Fig. 9.7.9 Calculating $K^{y}(\hat{\mathbf{y}}_t, \mathbf{y}_i)$

	A	B	C	D
11		K_y		
12	0.244592	1	2	3
13	1	0.8323824	0.244592097	0.2445921
14	2	0.2445921	0.244592097	0.83238237
15	3	0.8323824	0.244592097	0.2445921
16	4	0.2445921	0.83238237	0.2445921
17	5	0.8323824	0.244592097	0.2445921
18	6	0.2445921	0.244592097	0.83238237

Fig. 9.7.10 What if table

In figure 9.7.9, you can see two indexes. The label will have the index of Output label matrix which we got in step1. Hence it can be only varied from 1 to 3 because we have three output classes. The training data label will have the index of training data. It will be varied from 1 to 24. I1 to I3 cells will have the output label vector and b1 to b3 will have the binary code for the training data. We have again used a RBF kernel with σ=0.5. It is written in cell L7. The what-if table for $K^{y}(\hat{\mathbf{y}}_t, \mathbf{y}_i)$ will have 24 rows symbolizing 24 training data and three columns symbolizing three output classes. The first cell of what-if table will refer the kernel function cell which is L7. Now we can run the what-if analysis and get $K^{y}(\hat{\mathbf{y}}_t, \mathbf{y}_i)$.

Step8.2 Calculating $K^{\phi}(\mathbf{x}_i, \mathbf{x})$

For calculating it, we require a new x.

	D	E	F	G	H
40		**X1**	**X2**	**X3**	**X4**
41	**Test Data**	-1.198957881	-0.97894501	-0.97895	-0.97895

Fig. 9.7.11 Test data

Figure 9.7.11 shows the feature value of the new data point. $K^{\phi}(\mathbf{x}_i, \mathbf{x})$ is actually the kernel value calculated for each training data point with respect to the new test data 'x'. The kernel is RBF with $\sigma = 0.5$. The formula used is:-

=EXP(-0.5*(($E$41-Input!M9)^2+($F$41-Input!N9)^2+($G$41-Input!O9)^2+($H$41-Input!P9)^2)).

Here Range $E$41 to $H$41 refers to the test data point. Range Input!M9 to Input!P9 refers to the feature vector of the training data. This will calculate the kernel value with respect to the first data point. When you drag this formula down, you will get kernel value w.r.t. second training data and so on.

	A	B
45	**Index**	**Kx**
46	1	1
47	2	0.1470965
48	3	0.1470965
49	4	0.0216374
50	5	0.1470965
51	6	0.0216374
52	7	0.0216374
53	8	0.0031828
54	9	0.4873611
⋮	⋮	⋮

Fig. 9.7.12 Calculating $K^{\phi}(\mathbf{x}_i, \mathbf{x})$ for all value of i

For the user's convenience, we have copied the α vector (step7) to prediction sheet cell range C46:C69.

Step8.3 Calculating decision function

$$d(x) = \arg \max_{t=1,\ldots,T} \sum_{i=1}^{m} \alpha_i K^{y}(\hat{\mathbf{y}}_t, \mathbf{y}_i) K^{\phi}(\mathbf{x}_i, \mathbf{x}) \tag{9.7.4}$$

The summation shown in equation 9.7.4 is calculated for different output label 't' and whichever summation has the max value, the corresponding label is predicted. Since we have three classes, in excel we take three columns. The formula written is =$C46*$B46*B13. Here C46 is the cell having α, B46 is having $K^{\phi}(\mathbf{x}_i, \mathbf{x})$ values and B13 contains $K^{y}(\hat{\mathbf{y}}_t, \mathbf{y}_i)$ values. Drag this formula across all three classes till the number of training data points. Figure 9.7.13 will explain this.

	D	E	F	G
45		**Class1**	**Class2**	**Class3**
46		=$C46*$B46*B13	0.02019778	0.02019778
47		0.003018323	0.003018323	0.01027179
48		0.010109327	0.002970584	0.002970584
49		0.000444616	0.001513091	0.000444616
50		0.01010967	0.002970685	0.002970685
51		0.000443097	0.000443097	0.001507923
52		0.001481725	0.000435399	0.000435399
		-	-	-
		-	-	-
66		0.000569074	0.00016722	0.00016722
67		2.52002E-05	2.52002E-05	8.576E-05
68		8.29156E-05	2.43644E-05	2.43644E-05
69		1.24801E-05	3.66723E-06	3.66723E-06
70	Sum	0.143544025	0.048112437	0.058639148
71				

Fig. 9.7.13 Calculating for all classes

In the last column we take the summation of the three classes. Here we used the formula =SUM() function.

The maximum value of the three summation will be the value of predicted class. In figure 9.7.13 you see that Class 1 is having the maximum value hence we predict the test data point as Class 1.

	D	E	F	G	H	I	J
40		**X1**	**X2**	**X3**	**X4**		**Prediction Class**
41	**Test Data**	-1.198957881	-0.97894501	-0.97894501	-0.97895		1

Fig. 9.7.14 Calculating the class value

The formula that we have used in Prediction Class cell is given as: -
=MATCH(MAX($E$70:$G$70),$E$70:$G$70,0)

In this formula the MAX function return the maximum of three summation values. The MATCH function returns the relative position of an item in an array that matches a specified value in a specified order. The specified value here is the value returned by the MAX function. Hence this formula will help us to know the index of the output class.

Well, if you want to check the accuracy of the training data then the following steps will help you. In figure 9.7.15, you can see that the cells X1 to X4 will be populated from the input training table based on the index cell B78. Again we will use what-if data table so that we can iterate over all the training data and display the results. From cell A83 to A106, we have written all the indexes of the training data. The cell B82 contains the prediction value for the current test data point. Now you can run the what-if analyzer and select the column input cell as B78. You will get the prediction for all the training data. Now to compare the prediction with the actual output label, in column C we copy the actual output labels and in column D we write the formula to check whether the prediction is correct or not. The formula used is
=IF(C83=B83,TRUE,FALSE)

	A	B	C	D	E	F
75	Testing the Training Data					
76						
77			X1	X2	X3	X4
78	Index	1	-1.198957881	-0.97894501	-0.97894501	-0.97894501
79						
80						
81		Prediction	Actual	True/False		
82		1				
83	1	1	1	TRUE		
84	2	3	3	TRUE		
85	3	1	1	TRUE		
86	4	2	2	TRUE		
87	5	1	1	TRUE		
88	6	3	3	TRUE		
89	7	1	1	TRUE		
97	15	1	1	TRUE		
98	16	1	1	TRUE		
99	17	1	1	TRUE		
100	18	1	1	TRUE		
101	19	1	1	TRUE		
102	20	2	2	TRUE		
103	21	1	1	TRUE		
104	22	3	3	TRUE		
105	23	1	1	TRUE		
106	24	1	1	TRUE		

Fig. 9.7.15 Testing the training data

9.8 Summary

In this chapter we have provided step-by-step implementation of data mining problems using Support Vector Machines in Microsoft Excel. We first introduced the basic problems like XOR, circle, Spiral, Sinc function with help of non linear kernels like polynomial and radial basis function and later explained some real life problems like Eigen ear, contact lens, time series analysis etc.

References

[1] K.P Soman., Loganathan.R., Ajay. V: *Machine Learning with SVM and Other Kernel Methods :* ISBN: 978-81-203-3435-9.

[2] *K.P. Soman.*, Shyam Diwakar., V, Ajay., "*Insight into Data Mining*. Theory and Practice with CD ROM", ISBN: 978-81-203-2897-6